



ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**

ДИПЛОМНА РАБОТА

**Тема: „Проектиране и реализация на Kubernetes
оператор за унифицирано управление на системи
за сигурност и устойчивост“**

Изготвил:

Иван Цветанов Николов

Фак. №:

371325022

Научен ръководител:

доц. д-р инж. Явор Томов

София, 2026

Съдържание

I. ОБЗОР ПО ЛИТЕРАТУРНИ ИЗТОЧНИЦИ	2
1. Kubernetes и операторният модел	2
2. Сигурност на контейнерите – заплахи и предизвикателства при инструментите	6
3. Trivy – Сканиране за уязвимости и неправилни конфигурации	9
4. Falco – Сигурност при изпълнение и откриване на аномалии	11
5. LitmusChaos – Chaos Engineering и валидиране на устойчивостта	14
6. Как трите инструмента работят съвместно	16
7. Обосновка за унифициран подход	17
II. АРХИТЕКТУРА И ИЗПОЛЗВАНИ ТЕХНОЛОГИИ	19
1. Архитектурен обзор и принципи на дизайн	19
2. Go – основен език за реализация	20
3. Kubebuilder и CRD-базираният архитектурен подход	22
4. Програмно управление на Helm пакети	25
5. Go и chi за изграждане на REST API	27
6. Angular 21 и Signals API	29
7. Kubernetes CRD модел като хранилище на състоянието	31
III. ПРАКТИЧЕСКА ИМПЛЕМЕНТАЦИЯ И АРХИТЕКТУРНО РАЗВИТИЕ НА СИСТЕМАТА	33
1. CRD-ориентирана архитектурна концепция	33
2. Процес на реконсилация на CRD (Tools, Policy, PolicyReport)	38
3. Интеграция на Trivy	44
4. Интеграция на Falco	49
5. Интеграция на Litmus	52
6. REST API и Уеб табло	58
IV. ЗАКЛЮЧЕНИЕ, ОГРАНИЧЕНИЯ И БЪДЕЩА РАБОТА	72
1. Обобщение на постигнатите резултати	72
2. Архитектурни приноси	73
3. Ограничения	74
4. Бъдеща работа	75
5. Заключителни бележки	76
V. ИЗПОЛЗВАНИ ЛИТЕРАТУРНИ ИЗТОЧНИЦИ	78
VI. ПРИЛОЖЕНИЕ	79
Приложение 1a – Изходен код за реконсилация на инструмент (Tool)	79
Приложение 1b – Изходен код за реконсилация на политика (Policy)	83
Приложение 2 – Входна точка на eirenyx-api	87

I. ОБЗОР ПО ЛИТЕРАТУРНИ ИЗТОЧНИЦИ

1. Kubernetes и операторният модел

Kubernetes представлява система с отворен код за оркестрация на контейнери, първоначално разработена от Google и предоставена на Cloud Native Computing Foundation (CNCF) през 2015 г. [1,2] Архитектурата и концептуалният модел на платформата са пряко вдъхновени от вътрешната система за управление на клъстери на Google – Borg, която в продължение на повече от десетилетие управлява десетки хиляди работни натоварвания в мащабна продукционна среда. В резултат на това Kubernetes постепенно се утвърждава като де факто стандарт за разгръщане, мащабиране и управление на контейнеризирани приложения в cloud-native среди. В основата на Kubernetes стои модел, при който състоянието на разпределена система се представя като съвкупност от ресурси – типизирани и версионирани обекти, съхранявани в разпределено key-value хранилище (etcd). API сървърът изпълнява ролята на централен и единствен авторитетен интерфейс, чрез който всички компоненти на системата – както вътрешни контролери, така и външни клиенти – осъществяват операции по четене и запис върху тези обекти. Директен достъп до etcd не е допустим; всички промени в състоянието преминават през API сървъра, който прилага механизми за валидация, контрол на достъпа и оптимистична конкурентност.

Оперативен принцип в Kubernetes е т.нар. модел на желаното състояние (desired state). Потребителят декларира желаното състояние на системата (например Deployment с определен брой реплики или Service, който маршрутизира трафик към даден порт), а платформата непрекъснато се стреми да приведе реалното състояние в съответствие с декларираното. Това се реализира чрез набор от контролери — дългосрочно работещи процеси, които наблюдават определен набор от ресурси и предприемат коригиращи действия при отклонение между текущото и желаното състояние. Този декларативен подход освобождава потребителя от необходимостта да управлява директно преходите между състоянията, като вместо това той описва крайния резултат, който системата трябва да постигне. Всяка промяна в конфигурацията се разглежда като ново желано състояние, към което системата автоматично се адаптира чрез процес на непрекъсната реконсiliaция. Този модел идва с динамично и автоматизирано управление на ресурси, включително мащабиране, възстановяване при отказ и обновяване на приложения без прекъсване на услугата. Моделът улеснява изграждането на самовъзстановяващи се системи, тъй като при отклонение от желаното състояние контролерите автоматично предприемат действия за възстановяване на консистентността. В резултат Kubernetes осигурява висока степен на автоматизация и надеждност при управление на разпределени приложения.

Основният механизъм, чрез който работят контролерите, е т.нар. цикъл на реконсiliaция (reconciliation loop), който може да бъде формализиран по следния начин:

```
while true:
    desired = read desired state from API
    actual = observe current state of cluster
```

```
if actual != desired:  
    take action to move actual toward desired
```

Този модел се определя като ниво-ориентиран (level-triggered), не събитийно-ориентиран (edge-triggered). Контролерите не реагират директно на отделни събития, а на текущото състояние на системата. В резултат на това процесът на реконсiliaция е по своята същност идемпотентен – многократното извикване на функцията за реконсiliaция при едно и също желано състояние води до идентичен резултат. Това свойство е критично за устойчивостта на системата: при срыв и последващо рестартиране контролерът може да възстанови работата си, като просто прочете текущото състояние и продължи изпълнението, без необходимост от възпроизвеждане на предходни събития. Ниво-ориентираният подход опростява логиката на контролерите, тъй като елиминира необходимостта от проследяване и съхранение на събитийна история. Това намалява риска от натрупване на несъответствия при пропуснати или дублирани събития, което е често срещан проблем при събитийно-ориентираните системи. Моделът позволява естествена мащабируемост, тъй като множество контролери могат паралелно да извършват реконсiliaция върху различни ресурси без необходимост от централизирана координация. В крайна сметка този подход допринася за изграждането на устойчиви, самовъзстановяващи се системи, които са способни да поддържат консистентност дори при частични откази на компоненти.

Kubernetes предоставя набор от вградени контролери, като DeploymentController, ReplicaSetController, ServiceController, NodeController и други, всеки от които отговаря за управлението на конкретен тип ресурс. Въпреки че тези контролери покриват основните инфраструктурни сценарии, те не предоставят възможност за директно разширяване с логика, специфична за дадено приложение или домейн.

Потребителски ресурси и API за разширение

Основният механизъм за разширяване на Kubernetes е Custom Resource Definition (CRD). Това представлява декларация на схема, регистрирана в API сървър, която дефинира нов тип ресурс със собствена група, версия и вид (group, version, kind). След регистриране API сървърът автоматично осигурява валидиране на данните (чрез OpenAPI v3), стандартни REST операции (list, get, create, update, delete, watch) и съхраняване на обектите в etcd, наред с вградените ресурси. В резултат на това ресурсите, базирани на CRD, се третират като пълноправни елементи на Kubernetes екосистемата. Те могат да бъдат управлявани чрез kubectl, наблюдавани от informer механизми, използвани в owner references, ограничавани чрез RBAC политики и обработвани от контролери. [1]

От гледна точка на потребителя взаимодействието с персонализиран ресурс не се различава от това с вграден ресурс. CRD предоставя възможност за въвеждане на домейн-специфична логика в Kubernetes, без необходимост от промени в ядрото на системата, като чрез тях се изграждат разширения, които моделират сложни приложения и процеси като нативни Kubernetes ресурси. Чрез комбинирането на CRD с контролери се реализира т.нар. операторен модел, при който специфична бизнес логика се автоматизира и управлява декларативно. По този начин Kubernetes се установява в бизнес света като платформа за оркестрация на контейнери и в универсална система за управление на разнообразни разпределени приложения и услуги.

В рамките на Kubebuilder framework, CRD се дефинират чрез анотирани Go структури, като инструментът controller-gen [6] автоматично генерира съответния YAML манифест:

```
// +kubebuilder:object:root=true
// +kubebuilder:subresource:status
type Tool struct {
    metav1.TypeMeta    `json:",inline"`
    metav1.ObjectMeta `json:"metadata,omitzero"`
    Spec    ToolSpec    `json:"spec"`
    Status  ToolStatus `json:"status,omitzero"`
}
```

Основен архитектурен елемент при CRD е разделението между „spec“ и „status“. Полето „spec“ описва желаното състояние и се управлява от потребителя, докато „status“ отразява наблюдаваното състояние и се поддържа от контролера. Това разделение се реализира чрез отделни API подресурси и гарантира, че конкурентни операции от страна на потребителя и контролера не водят до взаимно презаписване на данни. Така се осигурява ясно разграничение между декларативната конфигурация и текущото състояние на системата, което е фундаментален принцип в Kubernetes архитектурата.

На практика, потребителят извършва промени единствено върху „spec“ чрез операции като „kubectl apply“, без да има директен достъп до „status“. От своя страна контролерът актуализира „status“ чрез специализирания „/status“ подресурс, като записва информация за текущото състояние - например фази на изпълнение, условия (conditions), грешки или метрики. Жизненият цикъл на ресурса става по-проследим и улеснява диагностицирането на проблеми в системата. Освен това разделението подпомага реализирането на идемпотентни контролери, тъй като те сравняват „spec“ и „status“, за да определят необходимите действия. При по-сложни оператори, „status“ често съдържа структурирана информация за прогреса на операции като инсталация, обновяване или изтриване на ресурси и се създава ясен и устойчив механизъм за синхронизация между декларативните намерения на потребителя и реалното състояние на системата.

Операторен модел (Operator Pattern)

Формализиран от CoreOS през 2016 г., операторният модел представлява архитектурен подход, при който „Custom Resource Definition“ (CRD) се комбинира с персонализиран контролер в единна логическа единица, реализираща т.нар. „оперативно домейн-специфично знание“. Аналогията с човешки оператор не е случайна – както администратор на база данни притежава експертни знания за инсталация, конфигурация, мащабиране, архивиране и възстановяване, така и Kubernetes операторът енкапсулира тези процеси под формата на програмна логика. Съществената разлика между обикновен контролер и оператор се състои в нивото на сложност и контекст, който се обработва в процеса на реконсiliaция. Докато базовият контролер може да създаде ресурс (например „Deployment“) при наличие на определен

CRD, операторът разбира пълния жизнен цикъл на управляваната система. Това включва зависимости между версии, правила за безопасно обновяване (например задължително преминаване през междинна версия), специфични процедури при отказ, както и последователността на възстановителните действия. В практиката операторите се използват широко за управление на сложни и състояние-зависими системи. При бази данни (напр. PostgreSQL, MongoDB, Redis) те управляват избор на главния нод, синхронизация на реплики и автоматизирано архивиране. При системи за съобщения като „Apache Kafka“ операторите реализират управление на теми, потребители и безпроблемни обновявания. В областта на мрежовата инфраструктура оператори като „cert-manager“ автоматизират издаването и подновяването на TLS сертификати, докато в сферата на сигурността инструменти като „trivy-operator“ интегрират сканиране за уязвимости чрез „Kubernetes-native“ ресурси. [3] Зрелостта на даден оператор се оценява чрез „Operator Capability Model“, който дефинира пет нива:

- Level 1 – Basic Install: Операторът може да инсталира управляваното приложение или инструмент в Kubernetes клъстера, но не поема по-сложни операции по неговия жизнен цикъл.
- Level 2 – Seamless Upgrades: Операторът поддържа контролирано и безопасно обновяване на приложението между версии, като намалява риска от прекъсване на услугата.
- Level 3 – Full Lifecycle: Операторът управлява пълния жизнен цикъл на приложението, включително инсталация, конфигурация, обновяване, наблюдение, възстановяване и премахване.
- Level 4 – Deep Insights: Операторът предоставя разширена наблюдаемост върху състоянието на приложението чрез метрики, логове, статуси, предупреждения и диагностична информация.
- Level 5 – Auto Pilot: Операторът може автономно да взема решения и да извършва оптимизации, мащабиране, възстановяване и самостоятелна реакция при проблеми без човешка намеса.

Kubebuilder и controller-runtime

Kubebuilder представлява стандартната рамка за разработка на Kubernetes оператори на езика Go, поддържана от Kubernetes SIG API Machinery. Тя предоставя автоматизирани средства за инициализация на проектна структура, включително генериране на директории, конфигурационни файлове (go.mod, Makefile) и начални шаблони за CRD и контролери. Чрез използването на т.нар. „marker annotations“ (напр. +kubebuilder:subresource:status, +kubebuilder:validation:Enum) се дефинират валидационни правила и RBAC политики, които впоследствие се генерират автоматично чрез инструмента controller-gen. [6]

В основата на Kubebuilder стои библиотеката controller-runtime, която предоставя абстракции за изграждане на контролери и управление на жизнения цикъл на оператора. Централният компонент е Manager, който отговаря за инициализацията на контролерите, конфигурирането на кеширащ клиент, управлението на лидер избор (leader election) и стартирането на health/readiness endpoints. Това позволява изграждане на високо-достъпни оператори, при които само една инстанция активно изпълнява логиката на реконсилация. Ключов е интерфейсът Reconciler, който дефинира метода „Reconcile(ctx context.Context, req Request) (Result, error)“. Той получава минимална информация (име и namespace на ресурса), след което извлича актуалното му състояние чрез клиента. Този pull-based модел гарантира, че контролерът винаги работи с

последното състояние на системата, независимо от последователността на събитията. Клиентът, от своя страна, използва кеширане чрез informer механизми, което значително намалява натоварването върху API сървъра при големи клъстери.

Механизмът на finalizers осигурява контролирано изтриване на ресурси. При наличие на finalizer, Kubernetes маркира ресурса за изтриване чрез DeletionTimestamp, но не го премахва физически, докато контролерът не изпълни необходимите операции по почистване (cleanup). Това гарантира, че външни зависимости - като създадени ресурси, файлове или конфигурации - се освобождават коректно преди окончателното премахване на обекта. [1]

2. Сигурност на контейнерите – заплахи и предизвикателства при инструментите

Проблемно пространство на сигурността при контейнерите

Преходът към контейнеризирани, cloud-native натоварвания въвежда съществено различен ландшафт на заплахите в сравнение с традиционните виртуални машини. В среда, базирана на виртуални машини, границата между натоварванията се осигурява на ниво хипервайзор; компрометирана виртуална машина не може директно да засегне други без експлоатация на уязвимости в самия хипервайзор. При контейнерите, обаче, всички процеси споделят ядрото на хост операционната система. Процес, изпълняван в контейнер, и процес на самия хост използват една и съща системна повикваща интерфейсна повърхност към ядрото, което води до съществени последици за сигурността. Проблемното пространство на сигурността при контейнерите обхваща три основни фази от жизнения цикъл на натоварването, всяка със специфични модели на заплахи, механизми за детекция и изисквания към инструментите:

- Сигурност на веригата на доставки и преди внедряване (pre-deployment): разглежда рисковете, вложени в артефактите преди тяхното разгръщане. Контейнерният образ представлява OCI-съвместим архив от файлови слоеве, изграден върху базов образ (най-често Linux дистрибуция) и допълнителни слоеве със специфични за приложението зависимости. Всеки слой може да съдържа операционни пакети, runtime среди и библиотеки, които потенциално включват известни уязвимости, регистрирани в „National Vulnerability Database“ (NVD) или в специфични за доставчика бази. Анализ на Aqua Security от 2023 г. [9] показва, че над 60% от образите в публични регистри съдържат поне една критична или високорискова CVE уязвимост. Рискът се усилва от практиката образите да не се обновяват регулярно, което води до натрупване на новооткрити уязвимости. Освен CVE, контейнерните образи могат да съдържат вградени тайни (API ключове, пароли, частни ключове), чувствителни конфигурационни файлове с неправилни права за достъп или бинарни файлове с ненужни setuid права. Kubernetes манифестите също въвеждат рискове: изпълнение на контейнери с root права, разрешена ескалация на привилегии, използване на hostPath томовете или прекомерно широки Linux capabilities увеличават потенциалния обхват на компроетиране.
- Сигурност по време на изпълнение (runtime security): обхваща заплахите, които възникват след стартиране на натоварването. Дори напълно актуализиран и коректно конфигуриран контейнер може да бъде компроетиран чрез

уязвимости на ниво приложение – SQL injection, remote code execution, десериализационни атаки, които позволяват на атакуващия да придобие контрол върху процеса в контейнера. След компрометиране, атакуващият може да опита ескалация на привилегии (например чрез kernel уязвимости), странично придвижване към други подове или namespace-и, изтичане на данни или използване на ресурси за криптомайнинг. Системи за „admission control“ като „OPA/Gatekeeper“ и „Kyverno“ адресират част от тези рискове, като налагат политики при създаване на ресурси, например предотвратяват стартиране на подове с небезопасни конфигурации. Тяхното ограничение е, че работят само с информацията, налична към момента на внедряване, и не могат да откриват аномалии, възникващи по време на изпълнение.

- Оперативна устойчивост (operational resilience): представлява третото измерение на сигурността. Система, която преминава всички проверки преди внедряване и не проявява аномалии по време на изпълнение, може въпреки това да не отговори на изискванията за надеждност при реални условия – мрежови прекъсвания, откази на възли, изчерпване на ресурси или недостъпност на зависимости. Сигурността и надеждността са взаимосвързани: система, неспособна да се възстановява, е уязвима към отказ на услуга (DoS) и оперативни сринове със сериозни последици за сигурността.

Проблемът с оперативната фрагментация

Екосистемата от инструменти за сигурност в Kubernetes се развива динамично и предлага решения за всяка от описаните фази. Основният проблем е липсата на независима еволюция, при която всеки инструмент използва собствен:

- Модел за инсталация: Helm чартове с различни схеми за конфигурация, различни RBAC изисквания и специфични namespace конвенции.
- Формат за конфигурация: „Trivy“ използва YAML или CLI параметри; „Falco“ дефинира правила чрез собствен DSL с макроси и списъци; „Litmus“ използва CRD обекти като „ChaosEngine“ и „ChaosExperiment“.
- Модел за отчетност: „Trivy“ генерира „VulnerabilityReport“ и „ConfigAuditReport CRD“ обекти или JSON/SARIF файлове; „Falco“ генерира структурирани JSON събития; „Litmus“ създава „ChaosResult CRD“.
- Оперативен жизнен цикъл: всеки инструмент изисква отделни процедури за обновяване, управление на тайни и конфигурации за мониторинг и алармиране.

Това води до значителна сложност в управлението. Екипите трябва да поддържат експертиза в множество различни технологии, а новите членове трябва да усвоят различни конфигурационни езици, оперативни процедури и интерфейси. При откриване на проблем, корелацията между резултати от различни инструменти изисква ръчно съпоставяне на несъвместими модели на данни. Този проблем е емпирично потвърден. Докладът на „CNCF“ за сигурност в „cloud-native“ среди от 2023 г. [2] идентифицира сложността на инструментите и интеграционния overhead като основна пречка пред последователното прилагане на политики за сигурност. Организациите посочват сигурността като втория най-значим източник на оперативно натоварване след наблюдаемостта.

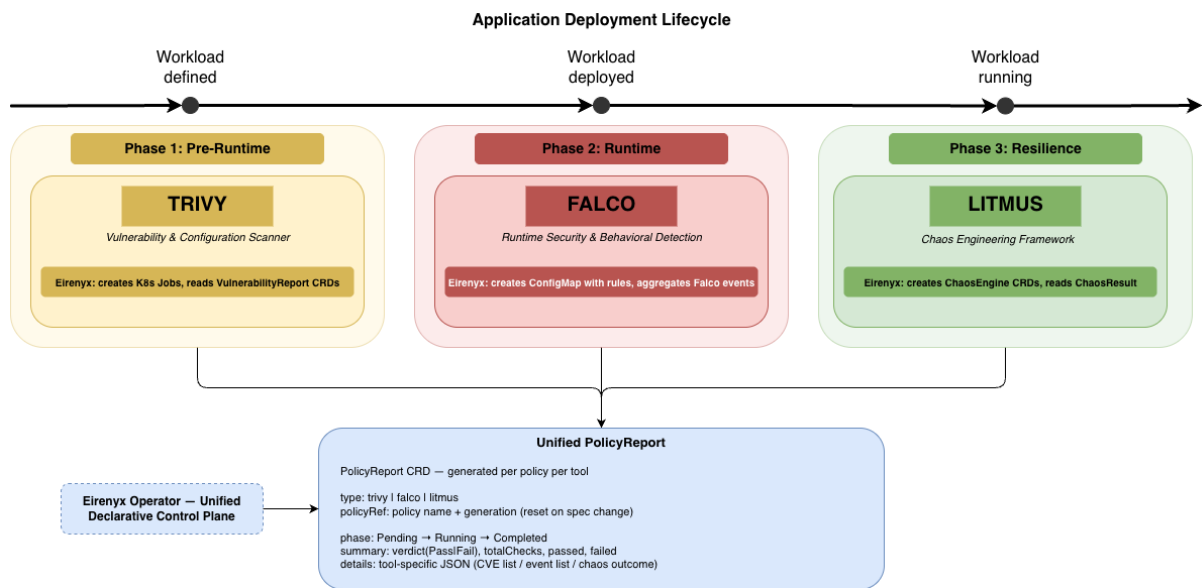
Съществуващи подходи за интеграция и техните ограничения

Съществуват различни подходи за интеграция на инструменти за сигурност, както в комерсиалния, така и в open-source контекста:

- Комерсиални платформи: като „Aqua Security“, „Sysdig Secure“ и „Prisma Cloud“ предоставят унифицирани интерфейси за агрегиране на резултати от различни инструменти. Въпреки тяхната функционалност, те имат три основни ограничения:
 - те са проприетарни (липсва прозрачност и възможност за разширение);
 - често са cloud-базирани (с риск за регулаторни изисквания за данни);
 - водят до „vendor lock-in“.
- Open-source решения: предлагат препоръчителни архитектури и практики, но не осигуряват цялостна автоматизация. Те дефинират концепции за интеграция, но оставят реализацията изцяло на потребителя.
- GitOps подходи (напр. ArgoCD, Flux): улесняват управлението на жизнения цикъл на инструментите, но не решават проблема с унифицирането на конфигурацията и отчетността. Всеки инструмент остава изолиран компонент със собствена логика и интерфейс.

В контекста на разгледаните предизвикателства възниква необходимостта от решение като Eirinux – open-source, Kubernetes-native оператор, който предоставя унифициран декларативен интерфейс за управление на сигурността. Подходът обхваща целия жизнен цикъл на инструментите, от тяхната инсталация и конфигурация до управление на политики и централизирана отчетност, без да заменя използваните технологии или да налага зависимост от проприетарни платформи. Реализацията на подобна архитектура изисква внимателен подбор на инструменти, които покриват различните аспекти на сигурността и същевременно могат да бъдат интегрирани в Kubernetes-native среда. В настоящата разработка са избрани три решения: Trivy, Falco и LitmusChaos, които адресират съответно сигурността на ниво артефакти (pre-deployment), поведението на системата по време на изпълнение (runtime) и устойчивостта при откази (resilience). Техният избор се базира на доказаната им зрялост, широко приложение в „cloud-native“ екосистемата и наличието на стандартизирани механизми за интеграция чрез оператори и CRD ресурси.

За да се разбере по-добре тяхната роля, процесът на разработка и внедряване на приложение следва да се разглежда като последователност от взаимосвързани етапи, всеки от които въвежда специфични рискове. Този процес включва изграждане на артефакти (build phase), внедряване в среда (deployment phase), изпълнение на приложението (runtime phase) и поведение при откази или натоварване (failure/stress phase). Всеки от тези етапи представлява потенциална точка на компрометиране и изисква прилагане на подходящи механизми за сигурност. Фигура 1 илюстрира този жизнен цикъл на приложението и показва как избраните инструменти осигуряват покритие върху отделните му фази. Диаграмата представя връзката между етапите на жизнения цикъл и съответните механизми за защита, като подчертава необходимостта от многослоен подход към сигурността.



Фигура 1: Цикъл на изграждане на приложение

От представения модел се вижда:

- Trivy: анализира контейнерните образи преди тяхното внедряване. Чрез идентифициране на известни уязвимости и конфигурационни слабости, инструментът предотвратява навлизането на компрометирани артефакти в продукционната среда. Това осигурява първия защитен слой, насочен към статичните характеристики на приложението.
- Falco: оперира в етапа на изпълнение, където наблюдава поведението на работещите контейнери в реално време. Чрез анализ на системни повиквания и поведенчески модели, той позволява откриване на аномалии и потенциални атаки, които не могат да бъдат установени чрез статичен анализ. По този начин се осигурява контрол върху динамичния аспект на сигурността, свързан с реалното функциониране на системата.
- LitmusChaos: въвежда контролирани откази и натоварвания с цел валидиране на устойчивостта на системата и оценка на способността на приложението да се възстановява при неблагоприятни условия, което е от значение за надеждността, и за сигурността на системата.

Комбинираното използване на трите инструмента създава цялостен, многослоен модел за защита, който обхваща както статичните свойства на приложението, така и неговото поведение в реална експлоатация и при откази. Това минимизира рискът от пропуски в сигурността и се осигурява последователно покритие на всички етапи от жизнения цикъл на приложението.

3. Trivy – Сканиране за уязвимости и неправилни конфигурации

Trivy е „open-source“, универсален инструмент за сканиране на уязвимости и неправилни конфигурации, разработен от „Aqua Security“ и публикуван през 2019 г. [8,9] Той адресира фазата на сигурност преди внедряване (pre-deployment), като отговаря на въпроса: какви известни рискове са вложени в артефактите, изграждащи даденото приложение?

Обхватът на сканиране на Trivy включва шест типа цели:

- Контейнерни образи: OCI-съвместими образи, извлечени от произволен регистър.
- Файлови системи: произволни директории, включително разархивирани слоеве на образи.
- Git хранилища: изходен код и манифести на зависимости в локално копие на репозиторията.
- Kubernetes манифести и Helm chart-ове: YAML конфигурации за откриване на неправилни настройки.
- SBOM документи: Software Bill of Materials във формат CycloneDX или SPDX.
- AWS акаунти: неправилни конфигурации в cloud инфраструктура чрез Trivy AWS провайдера.

За откриване на уязвимости Trivy поддържа локална база данни (актуализирана ежедневно от „NVD“, „GitHub Security Advisories“ и източници на доставчици), която се съпоставя с метаданните на пакетите, извлечени от сканираните артефакти. Съпоставянето обхваща както системни пакетни мениджъри (apt/dpkg, apk, rpm/yum/dnf), така и езиково-специфични мениджъри (npm/yarn, pip/poetry, cargo, go modules, maven/gradle, composer, gem).

Техническа архитектура на Trivy – Kubernetes Trivy Operator

Процесът на сканиране в Trivy се състои от четири основни етапа, като се започва анализът на артефакта. Trivy извлича целевия артефакт и неговите компоненти. При контейнерен образ се извлича OCI манифестът, изтеглят се всички слоеве (tar архиви) и се реконструира файловата система. След това се извършва обход на файловата система за откриване на метаданни на пакети (напр. /var/lib/dpkg/status, node_modules/.package-lock.json, go.sum) и бинарни сигнатури. Следва етапът на справка в база данни. За всеки идентифициран пакет и версия Trivy извършва заявка към локалната база данни за уязвимости с цел намиране на съответстващи CVE записи. Базата съдържа съпоставяне между (екосистема, име на пакет, диапазон от версии) и CVE идентификатори, оценки за сериозност (CVSS v2 и v3), EPSS (Exploit Prediction Scoring System) оценки и информация за налични поправки. Откриването на неправилни конфигурации се осъществява като при Kubernetes манифести и Helm chart-ове Trivy прилага набор от проверки, реализирани чрез Rego (езика за политики на OPA), както и вградени проверки. Те откриват често срещани проблеми като: контейнери, работещи с root права, липсващи resource лимити, привилегировани контейнери, host path mount-ове и липсващи security контексти. Процесът завършва с агрегиране и извеждане на резултати. Резултатите се агрегират, премахват се дублиранятията и се сериализират в избран формат (JSON, SARIF, табличен изглед, HTML, CycloneDX SBOM).

В Kubernetes клъстер, „Trivy“ се внедрява чрез „trivy-operator“, който автоматизира сканирането в рамките на клъстера. За разлика от използването като CLI инструмент в CI pipeline-и (което осигурява моментна снимка при build време), „trivy-operator“ осъществява непрекъснат мониторинг и сканира всички изпълнявани контейнерни образи. Това се реализира чрез:

1. Наблюдение на Pod и ReplicaSet ресурси за нови или обновени контейнерни образи.

2. Създаване на Kubernetes Job за всеки уникален образ, който трябва да бъде сканиран.
3. Изпълнение на Trivy контейнер в рамките на Job-а, който извършва сканиране и записва резултатите като JSON в стандартния изход.
4. Обработване на резултатите от оператора и създаване или обновяване на „VulnerabilityReport CRD“ в същия namespace като сканирания workload.

„VulnerabilityReport“ предоставя структурирано и достъпно за заявки представяне на резултатите от сканирането, което се съхранява в „etcd“ заедно с описвания „workload“. Това е интерфейсът, който се използва от „TrivyReportHandler“ в Eirenyx.

CVSS Модел за оценка на уязвимости

Trivy класифицира уязвимостите чрез „Common Vulnerability Scoring System“ (CVSS) – стандартизирана рамка за оценка на сериозността на уязвимости в сигурността. Оценка по CVSS v3 варира от 0 до 10 и се категоризират в пет нива представени в таблицата по долу:

CVSS Score	Сериозност	Типично въздействие
9.0–10.0	CRITICAL	Отдалечено изпълнение на код, пълен компромис на системата
7.0–8.9	HIGH	Значително изтичане на данни, ескалация на привилегии
4.0–6.9	MEDIUM	Ограничено въздействие, изисква специфични условия
0.1–3.9	LOW	Минимално въздействие, трудно за експлоатиране
N/A	UNKNOWN	Недостатъчна информация за оценка

В Eirenyx логиката за определяне на крайния резултат (verdict) по подразбиране класифицира CRITICAL и HIGH уязвимости като Fail. Това е съзнателен избор, съобразен с утвърдените практики в индустрията, според които уязвимости с такава сериозност представляват неприемлив риск за продукционни среди. Уязвимостите със средна (MEDIUM) и ниска (LOW) степен се отчитат, но не влияят върху крайния резултат, което намалява т.нар. „alert fatigue“ при инженерите, отговорни за платформата.

4. Falco – Сигурност при изпълнение и откриване на аномалии

Falco е „cloud-native“ проект за сигурност при изпълнение, първоначално разработен от „Sysdig“ през 2016 г. и дарен на „CNCF“ през 2018 г., като достига статус „graduated project“ през 2024 г. [2] Той адресира фазата на сигурност по време на изпълнение (runtime), като отговаря на въпроса: дали даден workload се държи очаквано по време на своето изпълнение?

За да се разбере необходимостта от Falco, е необходимо да се посочат ограниченията на механизмите за сигурност при admission. Admission контролерите, като „OPA/Gatekeeper“, „Kyverno“ и „Kubernetes PodSecurity admission controller“, оперират в момента на подаване на „pod“ спецификация към API сървър. Те могат да отхвърлят pod-ове с опасни конфигурации (привилегирани контейнери, hostPath mount-ове, липсващи security контексти) и да налагат политики при внедряване.

Въпреки това, те имат фундаментално времево ограничение: наблюдават единствено декларираното състояние на workload-a, но не и неговото поведение по време на изпълнение. Следните сценарии не могат да бъдат засечени чрез admission контрол:

- Контейнер с уеб приложение е допуснат с напълно коректна конфигурация, но впоследствие е компрометиран чрез SQL injection уязвимост, позволяваща изпълнение на shell команди. Контейнерът започва да създава shell процеси – ясен индикатор за компрометиране.
- Компрометиран контейнер започва да установява изходящи връзки към „command-and-control“ сървър през необичаен порт.
- Привилегирован контейнер (допуснат по легитимна причина, напр. мрежов плъгин) започва да чете чувствителни файлове извън предназначения си обхват.
- Зависимост с известна уязвимост за отдалечено изпълнение на код (RCE) се експлоатира по време на изпълнение за изтегляне и стартиране на злонамерен бинарен файл.

Всички тези поведения са невидими за admission политиките, тъй като възникват след допускането на workload-a. Сигурността при изпълнение изисква различен подход на непрекъснато наблюдение на реалното поведение на контейнерите.

Опериране на Kernel-Level повиквания

Falco функционира чрез прихващане на системни повиквания (syscalls) – интерфейсът, чрез който всички процеси (включително контейнеризирани) взаимодействат с Linux ядрото. Всяко действие на ниво процес със значение за сигурността включва „syscall“: създаване на файл (open, creat), стартиране на процес (execve, fork), установяване на мрежова връзка (connect, bind, accept) или достъп до паметта на ядрото (ptrace). Falco прихваща „syscalls“ чрез два механизма за инструментализация на ядрото:

- Kernel module: динамично зареждаем модул, който се интегрира с таблицата на системните повиквания чрез „tracpoint“ инфраструктурата. Този подход осигурява пълно покритие с минимален „overhead“, но изисква възможност за зареждане на kernel модули, което може да бъде ограничено в някои среди.
- eBPF probe: програма, компилирана до „eBPF“ байткод и зареждана в ядрото чрез системното повикване „bpf()“. eBPF програмите се изпълняват в защитена виртуална машина в ядрото и се верифицират предварително, за да се гарантира, че не могат да компрометират стабилността на системата. eBPF е наличен в Linux ядра 4.14+ и е предпочитаният подход в съвременни Kubernetes среди, тъй като не изисква kernel модул и работи в ограничени среди (напр. GKE, EKS с Bottlerocket).

И в двата случая Falco се изпълнява като „DaemonSet“, като се създава по един pod на всеки „node“, което гарантира наблюдение на всички контейнери в клъстера. Събраните „syscall“ събития се обогатяват с Kubernetes метаданни (име на pod, namespace, образ, labels) чрез заявки към Kubernetes API, след което се подават към механизма за правила.

Изпълнение на правила на Falco

Правилата са дефинирани чрез YAML-базиран домейн-специфичен език. Всяко правило съдържа:

- **condition:** логически израз върху полета от syscall събитията, който трябва да бъде изпълнен, за да се задейства правилото.
- **output:** шаблон за съобщение, описващо събитието.
- **priority:** ниво на приоритет (EMERGENCY, ALERT, CRITICAL, ERROR, WARNING, NOTICE, INFORMATIONAL, DEBUG).
- **tags:** етикети за категоризация и филтриране.

Правилата използват макроси (преизползваеми логически изрази) и списъци (набори от стойности), за да се избегне дублиране. Пример:

```
- macro: spawned_process
  condition: evt.type = execve and evt.dir = <
- list: shell_binaries
  items: [ bash, sh, zsh, ksh, fish ]

- rule: Terminal shell in container
  desc: A shell was spawned in a container
  condition: >
    spawned_process and
    container and
    proc.name in (shell_binaries) and
    not proc.pname in (shell_binaries)
  output: >
    Shell spawned in container (user=%user.name container=%container.name
    image=%container.image.repository:%container.image.tag
    shell=%proc.name parent=%proc.pname)
  priority: WARNING
  tags: [ shell, container ]
```

Falco предоставя стандартен набор от правила, поддържан от общността, който обхваща най-често срещаните модели на атаки. Потребителите могат да разширяват или модифицират тези правила. В контекста на Eirinux, компонентът FalcoEngine сериализира конфигурацията от „spec.falco“ в ConfigMap, който Falco зарежда като допълнителен файл с правила, позволявайки динамично прилагане без рестартиране на DaemonSet.

При задействане на правило, Falco генерира структуриран „alert“, съдържащ името на правилото, нивото на приоритет, изходното съобщение и всички релевантни полета от събитието. Тези известия могат да бъдат насочвани към множество изходи: стандартен изход (stdout), gRPC (за Falcosidekick), HTTP webhooks, Slack, PagerDuty и други. В контекста на Eirinux, този модел се използва от FalcoReportHandler, като броят на събитията, съответстващи на дадено правило в рамките на наблюдавания период,

определя крайния резултат (PolicyReport). Наличие на поне едно събитие означава, че правилото е било задействано, което индикира аномално поведение и води до резултат Fail.

5. LitmusChaos – Chaos Engineering и валидиране на устойчивостта

Chaos engineering представлява практика за целенасочено инжектиране на контролирани откази в дадена система с цел изграждане на увереност в нейната способност да устоява на нестабилни условия в продукционна среда. Дисциплината е въведена от „Netflix“ чрез инструмента „Chaos Monkey“, който произволно прекратява „EC2“ инстанции в продукцията, с цел да принуди разработчиците да изграждат услуги, устойчиви на откази. Основните принципи са формализирани в „Principles of Chaos Engineering“:

- Формиране на хипотеза за устойчиво (steady-state) поведение: дефиниране на нормалното състояние (напр. грешки $< 0.1\%$, латентност $p99 < 200\text{ ms}$).
- Вариране на реални събития: инжектиране на откази, които реално се наблюдават в продукцията (прекратяване на инстанции, мрежова латентност, откази на дискове).
- Изпълнение на експерименти в продукцията: тестове в staging среди не отразяват напълно реалното натоварване и топология.
- Автоматизация и непрекъснато изпълнение на експерименти: единичен експеримент дава моментна оценка, докато непрекъснатите експерименти осигуряват постоянна увереност.
- Минимизиране на обхвата на въздействие (blast radius): започва се с малки и контролирани експерименти, които постепенно се разширяват.

Значението на „chaos engineering“ за сигурността често се подценява. Дори система без известни уязвимости и без аномално поведение при изпълнение може да бъде уязвима към атаки, насочени към наличността (availability attacks) – състояния, които водят до отказ в обслужването на заявки. Ако дадена услуга спре да работи вследствие на изтриване на pod (което би трябвало да бъде обработено от self-healing механизмите на Kubernetes), атакуващият може да се възползва от това, например чрез заобикаляне на механизми за автентикация, нарушаване на логването или създаване на състезателни условия (race conditions) в критични кодови пътища.

CRD модел на LitmusChaos и архитектура на Kubernetes ниво

LitmusChaos е „CNCF“ проект в стадий „incubating“, който реализира „chaos engineering“ в Kubernetes-native среда. За разлика от по-ранни инструменти (Chaos Monkey, Gremlin), които оперират на ниво инфраструктура, Litmus моделира chaos експериментите като „Kubernetes Custom Resources“, което ги прави декларативни, проследими и съвместими с GitOps подхода.

Архитектурата на Litmus се базира на три основни CRD типа:

- ChaosExperiment: ресурс, който дефинира параметрите и логиката на изпълнение за конкретен сценарий на отказ. Litmus предоставя библиотека от готови експерименти, покриващи най-често срещаните сценарии:

Експеримент	Инжектиран отказ
pod-delete	Произволно изтриване на pod
pod-cpu-hog	Натоварване на CPU в контейнер
pod-memory-hog	Натиск върху паметта в контейнер
pod-network-latency	Изкуствено добавена мрежова латентност
pod-network-loss	Загуба на пакети в мрежовия интерфейс
node-drain	Cordoning и draining на Kubernetes node
disk-fill	Запълване на временната дискова памет

Всеки ChaosExperiment се параметризира чрез променливи на средата (напр. TOTAL_CHAOS_DURATION, CHAOS_INTERVAL, TARGET_PODS), което позволява повторното му използване в различни контексти.

- ChaosEngine: ресурс за изпълнение, който свързва ChaosExperiment с конкретен workload. Той съдържа:
 - appInfo: описание на целевия workload (namespace, label selector, тип ресурс).
 - engineState: active (стартиране на експеримента) или stop (прекратяване).
 - experiments: списък от експерименти с конкретни параметри.

Когато операторът на Litmus засече ChaosEngine в състояние active, той създава „chaos runner pod“, който изпълнява експеримента върху целевия workload.

- ChaosResult: резултатен ресурс, създаван от chaos runner-а след завършване на експеримента. Той съдържа резултата (Pass/Fail), както и метаданни за изпълнението (начален и краен момент, резултати от проверки)

В рамките на Eirenyx, компонентът LitmusEngine създава по един ChaosEngine CRD за всеки експеримент, дефиниран в списъка „spec.litmus.experiments“ на дадена Policy. От своя страна, LitmusReportHandler използва конфигурацията на експериментите, за да генерира PolicyReport, който отразява кои експерименти са били планирани и изпълнени. Този подход предоставя декларативен и ориентиран към политики интерфейс за управление на chaos експерименти. Вместо ръчно създаване на ChaosEngine ресурси, потребителят дефинира желаното поведение чрез Eirenyx Policy, а операторът автоматично управлява жизнения цикъл на съответните ресурси в клъстера.

6. Как трите инструмента работят съвместно

Трите разгледани инструмента – Trivy, Falco и Litmus, адресират различни аспекти от сигурността в рамките на жизнения цикъл на едно приложение в Kubernetes среда. Те оперират на три взаимно независими (ортогонални) нива, като всеки от тях покрива специфична фаза от жизнения цикъл на workload-а. Поради тази причина нито един инструмент не може да замени останалите, а тяхната реална стойност се проявява при комбинираното им използване в единна архитектура. Взаимодействието между инструментите може да бъде разгледано чрез времеви модел на жизнения цикъл на приложението, който включва етапите изграждане (build time), внедряване (deployment), изпълнение (runtime) и поведение при откази (stress/failure conditions). В този модел Trivy функционира в най-ранния етап, като анализира контейнерните образи преди тяхното внедряване. Основната му задача е да идентифицира известни уязвимости и неправилни конфигурации, като по този начин предотвратява навлизането на компрометирани артефакти в продукционната среда. Въпреки това, липсата на критични уязвимости не гарантира сигурността на системата по време на изпълнение. Falco допълва този модел, като осигурява наблюдение в реално време на поведението на изпълняващите се контейнери. Чрез анализ на системни повиквания на ниво ядро, той може да открие аномалии и индикатори за компрометиране, които не могат да бъдат засечени чрез статичен анализ. Това включва сценарии като стартиране на неочаквани процеси, установяване на необичайни мрежови връзки или достъп до чувствителни ресурси. Falco покрива динамичния аспект на сигурността, свързан с реалното поведение на приложението. Litmus разширява обхвата на сигурността, като въвежда измерение, свързано с устойчивостта на системата. Чрез контролирано инжектиране на откази, инструментът позволява да се оцени способността на приложението да функционира и да се възстановява при неблагоприятни условия. Този аспект е независим от наличието на уязвимости или аномалии — една система може да бъде напълно „сигурна“ от гледна точка на Trivy и Falco, но да се провали при реален отказ, което също представлява съществен риск за сигурността, особено по отношение на наличността и надеждността на критични услуги.

Въпреки тяхната допълваща се функционалност, трите инструмента не предоставят вградена интеграция помежду си. Всеки от тях използва собствен модел за конфигурация, различен формат за представяне на резултатите и отделни механизми за управление. Това води до фрагментация в процеса на анализ, при която инженерите трябва да работят с множество независими системи. За да се оцени цялостната сигурност на даден workload, е необходимо последователно изпълнение на сканиране с Trivy, наблюдение на Falco alerts и провеждане на експерименти с Litmus, последвано от ръчна корелация на резултатите. Тази липса на унифициран подход създава реални предпоставки за пропуски в сигурността. Когато информацията е разпределена между различни инструменти и не е обединена в единен модел, съществува риск отделни сигнали да бъдат интерпретирани изолирано и да не се разпознае тяхната комбинирана значимост. Например, наличие на уязвимост със средна степен, съчетано с аномално поведение при изпълнение и липса на устойчивост при отказ, може да индикира сериозен компромис, който остава незабелязан при отсъствие на централизирана корелация. В този контекст се откроява необходимостта от интеграционен слой, който да обедини трите инструмента в единна управляваща равнина. Именно този проблем адресира Eirinux, като предоставя декларативен интерфейс за управление и корелация на резултатите, което позволява цялостна оценка на сигурността на workload-а във всички фази от неговия жизнен цикъл.

7. Обосновка за унифициран подход

Описаната по-рано оперативна фрагментация не е резултат от недостатъци в самите инструменти. Trivy, Falco и Litmus са добре проектирани и ефективни в рамките на своето предназначение. Фрагментацията произтича от характеристиките на екосистемата, в която тези инструменти са разработени: независимо един от друг, за различни цели, в различни времеви периоди и без предварително дефиниран модел за интеграция. Операторният модел на Kubernetes предоставя подходяща абстракция за изграждане на унифицирана контролна равнина върху хетерогенни инструменти. Чрез използването на оператор могат да бъдат реализирани следните функционалности:

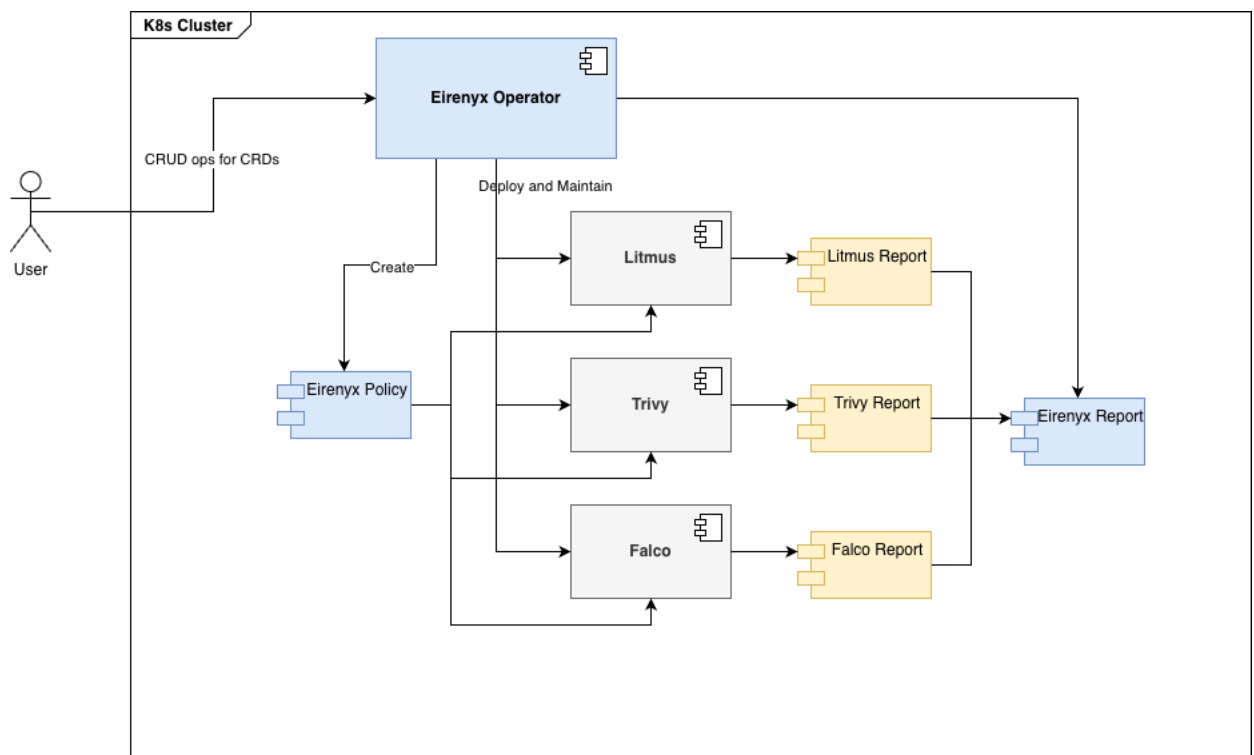
- Абстрахиране на инсталацията на инструментите чрез единен Tool CRD, което елиминира необходимостта от работа с различни Helm конфигурации.
- Унифициране на конфигурацията чрез централен Policy CRD, който предоставя общ интерфейс за дефиниране на политики за сигурност.
- Генериране на стандартизиран PolicyReport CRD, който агрегира резултатите от всички инструменти и позволява тяхната корелация.
- Управление на пълния жизнен цикъл на инструментите (инсталация, обновяване, мониторинг на състоянието и премахване), което осигурява последователност и намалява оперативната сложност.

Този подход позволява изграждането на централизирана управляваща равнина, която обединява различни инструменти в единна архитектура, без да нарушава тяхната автономност. На база на анализа на проблемната област могат да бъдат формулирани следните ключови изисквания към едно ефективно решение за унифицирано управление на сигурността:

- R1 – Неинвазивна интеграция: Решението трябва да управлява Trivy, Falco и Litmus без да ги модифицира или заменя. Всеки инструмент трябва да запази своята функционална независимост, като операторът добавя единствено управляващ слой.
- R2 – Декларативно управление на жизнения цикъл: Всички операции, свързани с инсталация, конфигурация, обновяване и премахване на инструментите, трябва да бъдат дефинирани чрез Kubernetes ресурси. Не се допуска използване на ръчни CLI команди или скриптове.
- R3 – Унифициран модел за политики: Политиките за сигурност трябва да могат да бъдат дефинирани чрез единен CRD интерфейс, независимо от конкретния инструмент. Това елиминира необходимостта от използване на множество конфигурационни езици.
- R4 – Нормализирана отчетност: Резултатите от различните инструменти трябва да бъдат представени в унифициран формат, който позволява автоматизирана обработка, корелация и визуализация.
- R5 – Kubernetes-native състояние: Цялото състояние на системата трябва да бъде съхранявано в Kubernetes ресурси. Не трябва да се използват външни бази данни или неустойчиво състояние извън API сървъра.
- R6 – Съвместимост с GitOps: Конфигурацията трябва да бъде изцяло декларативна и управляема чрез YAML манифести, съхранявани в Git хранилище и прилагани чрез инструменти като Flux или ArgoCD.

Eirenyx удовлетворява всички дефинирани изисквания чрез трислойна архитектура, базирана изцяло на Kubernetes-native принципи. Първият слой включва

Tool CRD и съответния ToolReconciler, които реализират изискванията R1 и R2. Чрез този механизъм жизненият цикъл на инструментите се управлява декларативно, като инсталацията и обновяването се извършват автоматизирано чрез Helm. Вторият слой е представен от Policy CRD и PolicyReconciler, както и специфичните за инструментите реализации на Engine интерфейси. Този слой адресира изискването R3, като предоставя единна точка за дефиниране на политики, съдържаща подспецификации за всеки инструмент. Третият слой включва PolicyReport CRD и съответните ReportHandler компоненти, които нормализират резултатите от различните инструменти в общ формат. Реализира се изискването R4, като се осигурява унифицирана оценка на състоянието чрез стандартен показател (Verdict) и структурирани детайли. Изискванията R5 и R6 са удовлетворени чрез изцяло декларативния характер на архитектурата и използването на Kubernetes CRD ресурси като единствен механизъм за съхранение на състояние и конфигурация. Цялостната архитектура на системата, включително взаимодействието между отделните компоненти и инструментите, е представена на Фигура 2.



Фигура 2: Цялостна архитектурна диаграма за Kubernetes операторът

II. АРХИТЕКТУРА И ИЗПОЛЗВАНИ ТЕХНОЛОГИИ

1. Архитектурен обзор и принципи на дизайн

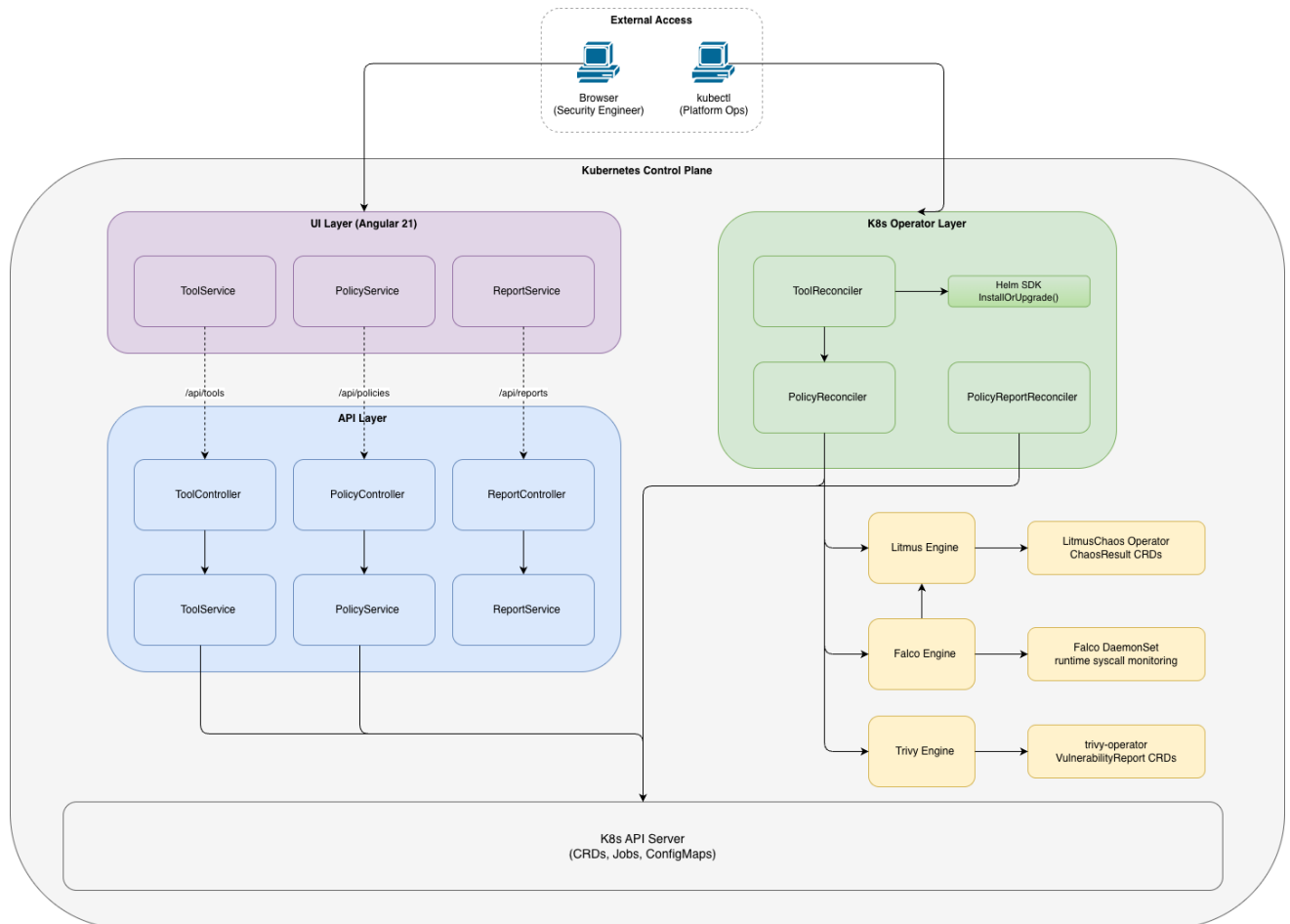
Системата Eirenyx е проектирана като съвкупност от три ясно разграничени и независимо деплойваеми компонента: Kubernetes оператор (eirenyx), бекенд API слой (eirenyx-api) и уеб потребителски интерфейс (eirenyx-ui). Всеки от тези компоненти е реализиран с различна технологична стекова основа и изпълнява строго дефинирана роля в цялостната архитектура.

Операторът „eirenyx“, реализиран на езика Go, представлява ядрото на системата. Той е отговорен за управлението на жизнения цикъл на външните инструменти (Trivy, Falco и Litmus), както и за изпълнението и наблюдението на дефинираните политики. Това е единственият компонент, който осъществява директна интеграция с тези инструменти, като по този начин изолира сложността на тяхната конфигурация и управление. Компонентът „eirenyx-api“, също реализиран на Go, функционира като REST API сървър и изпълнява ролята на посредник между потребителския интерфейс и Kubernetes API. Той представлява безсъстоятелен (stateless) слой, който трансформира HTTP заявки в операции върху Kubernetes ресурси и обратно. Осигурява се стандартизиран и контролиран достъп до системата, без директна зависимост на UI слоя от Kubernetes. Потребителският интерфейс „eirenyx-ui“, реализиран с TypeScript и Angular, предоставя визуална среда за управление и наблюдение на ресурсите. Той консумира данните, предоставени от API слоя, и ги визуализира по удобен за потребителя начин, без да съдържа бизнес логика или директни зависимости към Kubernetes.

Това архитектурно разделение следва принципа за единствена отговорност на ниво компонент (Single Responsibility Principle), част от по-известния SOLID подход за разработка на софтуер. Всеки компонент изпълнява точно определена функция, което води до намалена свързаност (coupling), по-лесно тестване и възможност за независимо развитие, подмяна или мащабиране на отделните части на системата. Като основен обединяващ принцип в архитектурата на Eirenyx е концепцията, че Kubernetes е единственият източник на истина (single source of truth). В системата не се използват външни бази данни, локални конфигурационни файлове или временни състояния в паметта, които не могат да бъдат възстановени. Цялото персистентно състояние се съхранява чрез Kubernetes Custom Resource обекти в etcd. Този подход има редица известни архитектурни и оперативни предимства. На първо място, операторът може да бъде рестартиран по всяко време без риск от загуба на състояние или повторно изпълнение на вече приключили операции, тъй като текущото състояние винаги се извлича от Kubernetes API. На второ, всички конфигурации са изразени чрез декларативни YAML манифести, което позволява тяхното съхранение в системи за контрол на версиите и управление чрез GitOps инструменти като Flux или ArgoCD. Допълнително, всяка промяна в състоянието се регистрира от audit логовете на Kubernetes API сървъра, което осигурява проследимост и контрол.

Друг съществен елемент е, че както API сървърът, така и потребителският интерфейс работят върху един и същ авторитетен източник на данни. Това елиминира класическия проблем с кеширането и синхронизацията на състоянието, тъй като няма отделни слоеве, които да поддържат независими копия на данните. Взаимодействието между трите компонента и Kubernetes API е илюстрирано на Фигура 3. Както е

показано, операторът комуникира директно с Kubernetes API за управление на ресурсите и интеграция с външните инструменти, докато API слойът осигурява посреднически достъп за потребителския интерфейс. UI компонентът от своя страна не комуникира директно с Kubernetes, а използва API слоя като единствен вход към системата.



Фигура 3: Интеграция между компонентите и интерфейсна карта на системата

Тази архитектурна организация осигурява ясно разделение на отговорностите, висока степен на модулност и устойчивост на системата, като същевременно създава стабилна основа за разширяемост и интеграция с допълнителни инструменти в бъдеще.

2. Go – основен език за реализация

Всички бекенд компоненти на системата Eirinux са реализирани на езика Go (версия 1.25). Този избор е направен целенасочено и е обусловен от изискванията на Kubernetes-native разработката, както и от характеристиките на самата екосистема. Цялата Kubernetes платформа е изградена върху Go. Основни компоненти като Kubernetes API сървър, kubelet, инструментът kubectl, както и ключови библиотеки като controller-runtime и client-go, са реализирани именно на този език. Същото важи и за утвърдени оператори в продукционна среда като Cert-Manager, Crossplane, Prometheus Operator и Argo CD. Това има директни последици върху начина на разработка. Kubernetes типовата система е естествено представена чрез Go типове.

Работата с Kubernetes ресурси в Go осигурява силно типизирана среда, при която се използва пълноценен компилаторен контрол върху данните. Това елиминира необходимостта от динамична обработка на JSON структури, runtime проверки и неявни преобразувания. В резултат, грешки като липсващи полета, неправилни типове или некоректно използване на API се откриват още на етап компилация, което значително повишава надеждността на системата. Допълнително предимство е наличието на богата и добре поддържана екосистема от библиотеки. Всички необходими инструменти за разработка на оператори – включително Kubernetes клиентите, Helm SDK и controller framework – са нативно реализирани на Go и се поддържат активно. За разлика от други програмни езици, където често се използват обвивки около REST API, Go предоставя директен и пълен достъп до функционалността на платформата.

От гледна точка на deployment и сигурност, Go предлага директна компилация до един самостоятелен бинарен файл. Не се изисква наличие на допълнителна среда за изпълнение, виртуална машина или динамично зареждане на зависимости, като така изграждането на минимални контейнерни образи, които съдържат единствено изпълнимия файл, намалява повърхността за атаки, тъй като липсват излишни компоненти като shell, пакетни мениджъри или системни инструменти, а размерът на образа остава изключително малък. Съществен аспект е и моделът за обработка на грешки в Go. Езикът използва експлицитен механизъм чрез връщане на стойност от тип error, което изисква всяка грешка да бъде обработена или изрично предадена нагоре по веригата от извиквания. Това е от значение при разработката на Kubernetes оператори, където в рамките на един reconciliation цикъл могат да бъдат извършени множество API операции. Експлицитното управление на грешките гарантира, че нито една грешка няма да бъде пропусната или игнорирана, за разлика от изключенията в езици като Java или Python, където е възможно те да се разпространяват без адекватна обработка.

Конкурентният модел на Go също е особено подходящ за операторния патерн. Framework-ът controller-runtime обработва всяка reconciliation заявка в отделна goroutine. Лекотата и ефективността на goroutine модела позволяват едновременното изпълнение на голям брой такива операции без значителен ресурсен разход. Освен това, механизмът context.Context предоставя стандартизиран начин за управление на жизнения цикъл на операциите, включително тяхното прекратяване при необходимост, което е критично при работа с външни ресурси и Kubernetes API.

Разгледани алтернативи

В процеса на проектиране са разгледани и други програмни езици, като всеки от тях е оценен спрямо изискванията за Kubernetes-native система.

- Python е широко използван език с богата екосистема, но липсата на силна типизация при работа с Kubernetes API води до повишен риск от runtime грешки. Освен това, операторни рамки като kopf са по-малко зрели в сравнение с controller-runtime, а самият език има по-високи изисквания към паметта и по-бавно стартиране.
- Java и JVM-базираните технологии предоставят силна типизация и стабилна екосистема, но са свързани със значителен ресурсен overhead. Стартирането на JVM е по-бавно, което влияе върху механизми като leader election, а контейнерните образи са значително по-големи, което е неблагоприятно за Kubernetes среди.

- Rust предлага отлична производителност и висока степен на безопасност, но екосистемата за работа с Kubernetes все още не е достатъчно зряла. Библиотеки като kube-rs предоставят базова функционалност, но изостават спрямо Go екосистемата по отношение на интеграция и поддръжка. Допълнително, езикът има по-висока сложност и по-стръмна крива на обучение.
- Node.js също е разгледан като възможност, но липсата на compile-time типова безопасност, зависимостта от V8 runtime и ограничената зрялост на операторните рамки го правят неподходящ избор за подобен тип система.

В резултат на този анализ, Go се утвърждава като най-подходящият избор, тъй като съчетава силна интеграция с Kubernetes, висока производителност, надеждност и минимални оперативни изисквания.

3. Kubebuilder и CRD-базираният архитектурен подход

Kubebuilder представлява официален framework за разработка на Kubernetes оператори на езика Go, поддържан от Kubernetes SIG API Machinery. Основната му стойност се състои в автоматизираното генериране на код, което значително намалява необходимостта от ръчно създаване на инфраструктурни компоненти. При традиционен подход разработчикът трябва самостоятелно да дефинира CRD схеми, да конфигурира наблюдение върху ресурси (watches), да управлява контролерната логика и да свърже всички компоненти в работещ процес. Kubebuilder автоматизира тези стъпки, като генерира необходимия код на база аотирани Go структури. [4] Създаването на базов проект изисква минимален брой команди и води до генериране на пълна проектна структура, включваща:

- дефиниции на API обектите (Spec и Status),
- автоматично генерирани помощни методи (напр. DeepCopy),
- контролерна логика с дефинирана точка за имплементация (Reconcile),
- Kubernetes CRD YAML файлове,
- RBAC конфигурации.

Разработчикът се фокусира единствено върху бизнес логиката – дефиниране на структурата на ресурсите и имплементация на reconciliation логиката. Всички останали аспекти, свързани с интеграцията в Kubernetes екосистемата, се поемат от toolchain-a.

CRD маркери – трансформация от Go структури към Kubernetes API

Kubebuilder използва специализирани аотации в Go, наречени markers, които служат като източник на информация за генериране на Kubernetes ресурси. [4] Чрез тях една Go структура може да бъде автоматично трансформирана в пълноценен Kubernetes API обект.

Тези маркери позволяват:

- дефиниране на валидиращи правила (напр. допустими стойности чрез Enum),
- задаване на стойности по подразбиране,
- конфигуриране на статус подресурси,
- дефиниране на колони за визуализация чрез kubect!,
- указване на поведение при сериализация.

```
// +kubebuilder:object:root=true
// +kubebuilder:subresource:status
// +kubebuilder:printcolumn:name="Type",type=string,JSONPath=`.spec.type`
//
+kubebuilder:printcolumn:name="Enabled",type=boolean,JSONPath=`.spec.enabled`
//
+kubebuilder:printcolumn:name="Healthy",type=boolean,JSONPath=`.status.healthy`
type Foo struct {
    metav1.TypeMeta   `json:",inline"`
    metav1.ObjectMeta `json:"metadata,omitzero"`
    Spec   FooSpec   `json:"spec"`
    Status FooStatus `json:"status,omitzero"`
}

type FooSpec struct {
    // +kubebuilder:validation:Enum=trivy;falco;litmus
    Type FooType `json:"type"`

    // +kubebuilder:default=false
    Enabled bool `json:"enabled"`

    // +optional
    Namespace string `json:"namespace,omitempty"`

    // +optional
    // +kubebuilder:pruning:PreserveUnknownFields
    Values runtime.RawExtension `json:"values,omitempty"`
}
```

Генерацият инструмент `controller-gen` обработва тези анотации и създава няколко ключови артефакта. [6] На първо място се генерира OpenAPI v3 схема, която се вгражда в CRD YAML файла. Тази схема се използва от Kubernetes API сървъра за валидация на ресурсите още при тяхното създаване. Например, ако е дефиниран ограничен набор от допустими стойности за дадено поле, API сървърът ще отхвърли всяка невалидна заявка, без тя да достигне до оператора. На второ място се генерират DeepCopy методи, които са необходими за коректната работа на кеширащия механизъм на `controller-runtime`. Те гарантират, че обектите могат да бъдат копирани безопасно, без странични ефекти. Допълнително се генерират RBAC конфигурации, които описват точно необходимите права за оператора, както и визуални колони за командата `kubectl get`, позволяващи по-информативно представяне на ресурсите. Практическият резултат от този подход е, че Go структурата се превръща в единствен източник на истина за API дефиницията. Не се изисква поддръжка на отделни YAML схеми, което елиминира риска от разминаване между код и конфигурация.

controller-runtime – механизъм за reconciliation

Библиотеката `controller-runtime` предоставя изпълнителната среда за Kubernetes контролери и реализира основния механизъм за reconciliation. Тя включва няколко ключови компонента, които осигуряват устойчиво и ефективно управление на състоянието. [5]

Мениджърът представлява централен координиращ компонент, който управлява жизнения цикъл на оператора. Неговите основни функции включват:

- инициализация на Kubernetes клиента и стартиране на informer механизми за наблюдение на ресурси;
- реализиране на leader election чрез Kubernetes ресурс от тип Lease, което гарантира, че само една инстанция на оператора изпълнява reconciliation логиката в даден момент;
- стартиране на всички контролери като паралелни процеси (goroutines);
- предоставяне на health и readiness HTTP endpoints;
- управление на коректно изключване на процеса при получаване на сигнал.

```
mgr, err := ctrl.NewManager(ctrl.GetConfigOrDie(), ctrl.Options{
    Scheme:                scheme,
    HealthProbeBindAddress: ":8081",
    LeaderElection:         true,
    LeaderElectionID:       "eirenyx-leader-election",
})
```

Клиентът, предоставен от controller-runtime, използва кеширащ механизъм, базиран на Kubernetes informers. [5] Това е стандартен подход при изграждане на Kubernetes оператори, тъй като позволява контролерът да наблюдава състоянието на ресурси в клъстера без постоянно да изпраща заявки към Kubernetes API сървър. Informer-ите работят чрез механизма list-watch: първоначално извличат списък с ресурси от даден тип, след което поддържат локален кеш чрез наблюдение на последващи промени.

На практика това означава, че голяма част от операциите за четене (Get и List) се изпълняват върху локална in-memory репрезентация на състоянието, а не чрез директна заявка към API сървър при всяко извикване. Този модел има няколко важни технически предимства:

- операциите за четене се извършват от локална памет, което намалява латентността;
- значително се редуцира натоварването върху Kubernetes API сървър;
- контролерът получава последователна представа за наблюдаваните ресурси в рамките на reconciliation цикъл;
- намалява се рискът от throttling при голям брой ресурси или чести reconcile събития.

Важно е да се подчертае, че кеширащият клиент не променя начина, по който се извършват операциите за запис. Операциите по създаване, обновяване и изтриване на ресурси (Create, Update, Patch, Delete) се изпращат директно към Kubernetes API сървър. Това гарантира, че всяка промяна в желаното или наблюдаваното състояние се записва в authoritative state store на Kubernetes, тоест в etcd, чрез API сървър.

Finalizer-ите представляват механизъм в Kubernetes, който позволява изпълнение на допълнителна логика преди окончателното изтриване на даден ресурс. Когато върху ресурс е зададен finalizer, Kubernetes не го премахва веднага при заявка за изтриване. Вместо това ресурсът се маркира с поле deletionTimestamp, което указва, че обектът е в процес на изтриване, но все още не е физически премахнат от API сървър. Контролерът засича това състояние по време на reconciliation и изпълнява

необходимите действия по почистване. В контекста на Eirenyx това включва премахване на външни или вторични ресурси, създадени при управлението на даден инструмент. Например при изтриване на Tool ресурс операторът трябва да премахне инсталираните чрез Helm ресурси, свързани със съответния инструмент. Едва след успешно приключване на тази cleanup логика контролерът премахва finalizer-а от ресурса. След премахването му Kubernetes може да завърши процеса по изтриване. Този механизъм е особено важен при оператори, които управляват ресурси извън самия custom resource. Без finalizer би било възможно Tool ресурсът да бъде изтрит от API сървър, докато реално инсталираните Helm release-и, service account-и, роли, deployment-и или други Kubernetes обекти останат в клъстера. Такива ресурси се наричат orphaned resources и могат да доведат до неконсистентно състояние, затруднена поддръжка и непредвидимо поведение при повторна инсталация. [1,7]

Kubernetes поддържа и отделен механизъм за дефиниране на зависимости между ресурси чрез ownerReferences. Те описват връзка на собственост между два Kubernetes обекта и позволяват изграждане на йерархична структура, управлявана от вградения garbage collector на Kubernetes. Когато родителският ресурс бъде изтрит, garbage collector-ът може автоматично да премахне всички зависими обекти, които го посочват като owner. В архитектурата на Eirenyx тази йерархия е организирана по следния начин:

- Tool ресурсът е родителският ресурс;
- Policy ресурсите са зависими от съответния Tool;
- PolicyReport ресурсите са зависими от съответните политики.

Тази структура отразява логическата зависимост между основните обекти в системата. Tool описва инсталацията и жизнения цикъл на конкретен инструмент, например Trivy, Falco или Litmus. Policy дефинира как този инструмент трябва да бъде използван, а PolicyReport съдържа резултатите от изпълнението на конкретната политика. Следователно политика без съществуващ инструмент няма самостоятелен смисъл, както и отчет без съответна политика не представлява валидно състояние на системата. При изтриване на ресурс от тип Tool, Kubernetes автоматично премахва свързаните Policy ресурси чрез механизма за garbage collection. От своя страна изтриването на политиките води до премахване на техните PolicyReport ресурси. Така операторът не е необходимо да реализира допълнителна ръчна логика за каскадно изтриване на тези обекти. Това намалява сложността на controller логиката и използва стандартен Kubernetes механизъм за поддържане на консистентност между свързаните ресурси.

4. Програмно управление на Helm пакети

Helm представлява пакетен мениджър за Kubernetes, който стандартизира процеса по инсталиране и управление на приложения в клъстерна среда. Основната му концепция е т.нар. Helm chart – версияно управляван, темплейтизиран набор от Kubernetes YAML манифести, описващ пълна инсталация на дадено приложение. Всеки chart включва всички необходими ресурси за функционирането на приложението, като например Deployments, Services, ServiceAccounts, ClusterRoles, ConfigMaps и Custom Resource Definitions (CRDs). Освен това, chart-ът предоставя конфигурационен файл values.yaml, който позволява параметризиране на инсталацията без необходимост от директна модификация на шаблоните. [7] Инструментите Trivy, Falco и Litmus,

използвани в рамките на Eirenyx, се разпространяват като официални Helm chart-ове, поддържани от съответните им разработчици:

Инструмент	Helm chart	Инсталирани ресурси
Trivy	<code>aquasecurity/trivy-operator</code>	Deployment, RBAC конфигурации, CRD за VulnerabilityReport
Falco	<code>falcosecurity/falco</code>	DaemonSet, kernel модул или eBPF probe, базови правила
Litmus	<code>litmuschaos/litmus</code>	Operator Deployment, CRD за ChaosEngine и др.

Използването на Helm за инсталиране на тези инструменти е утвърдена практика в Kubernetes екосистемата. Алтернативният подход – директно прилагане на YAML манифести – води до загуба на функционалности като управление на версии, контролирани обновления и възможност за параметризация на конфигурацията.

В архитектурата на Eirenyx се използва Helm Go SDK (`helm.sh/helm/v3`), вместо извикване на Helm CLI като външен процес. Този избор има съществено значение от гледна точка на сигурност, контрол и интеграция. Използването на Helm CLI предполага стартиране на външна команда чрез системни извиквания, което води до редица ограничения. Необходимо е наличието на Helm бинарен файл в контейнерния образ, което увеличава неговия размер и повърхността за атаки. Освен това, резултатите от изпълнението са налични единствено като текстов изход, което усложнява обработката на грешки и достъпа до състоянието на инсталацията. За разлика от това, Helm SDK предоставя програмно, типобезопасно API за управление на chart-ове. Чрез него операторът може директно да инициализира конфигурация, да стартира инсталация или обновяване и да получи структурирана информация за резултата. Състоянието на инсталирания release е достъпно чрез типизирани структури, а грешките се обработват като стандартни Go обекти и възможността за използване на т.нар. wait semantics. При инсталация операторът може да изчака до пълното стартиране на всички ресурси (напр. готовност на Deployment репликите) или до изтичане на зададен timeout. Това елиминира необходимостта от допълнителна логика за проверка на състоянието. Използването на Helm SDK води до по-компактна, сигурна и самостоятелна реализация, при която цялата логика е имплементирана в рамките на Go кода и подлежи на пълен контрол и одит.

Helm стойности и потребителска конфигурация

Конфигурацията на Helm chart-овете в Eirenyx се реализира чрез полето ToolSpec.Values, което е от тип runtime.RawExtension. Това представлява непрозрачен JSON обект, който се подава директно към Helm инсталацията като стойности за параметризация. Този подход позволява на платформените инженери да конфигурират детайлно поведението на инструментите, без да е необходимо Eirenyx да дефинира изрично всички възможни параметри. Например, при конфигуриране на Falco може да бъде избран eBPF драйвер вместо kernel модул, както и да бъдат активирани допълнителни механизми за извеждане на данни. Подадените от потребителя стойности

се комбинират с дефинираните по подразбиране конфигурации на Eirenuх, след което се предават към Helm SDK за изпълнение. Този дизайн следва принципа Open/Closed (отворен за разширение, затворен за модификация). Системата не изисква промени в кода при добавяне на нови конфигурационни параметри, но позволява гъвкаво разширяване чрез предоставяне на допълнителни стойности от страна на потребителя. [7]

5. Go и chi за изграждане на REST API

Компонентът eirenuх-api е проектиран като максимално опростен и ясно дефиниран слой в архитектурата. Неговата основна функция е транслация на HTTP заявки от потребителския интерфейс (Angular UI) към операции върху Kubernetes API, както и преобразуване на отговорите (включително грешки) обратно в стандартизирани HTTP отговори. API слойът не съдържа бизнес логика, не поддържа собствено състояние и не реализира самостоятелни механизми за автентикация или авторизация. Това архитектурно решение е съзнателно, тъй като Kubernetes API вече предоставя пълноценна инфраструктура за сигурност, включваща механизми за автентикация (service account токени, OIDC), авторизация (RBAC) и audit логване. Дублирането на тези механизми в отделен слой би довело до създаване на две независими системи за контрол на достъпа, което усложнява анализа, поддръжката и сигурността на системата. Вместо това, Eirenuх разчита изцяло на Kubernetes като централен орган за управление на достъпа. Съществен аспект на архитектурата е, че API слойът използва същите Go типове, дефинирани в оператора. Модулът eirenuх се използва като зависимост, което гарантира, че всяка промяна в CRD структурата води до компилаторни грешки в API слоя, ако не бъде отразена коректно. Това елиминира риска от несъответствие между представянето на ресурсите в различните компоненти на системата.

За реализиране на HTTP маршрутизацията в eirenuх-api е избран chi (версия 5.2.5). Изборът е базиран на няколко ключови архитектурни критерия. На първо място, chi има минимална зависимост от външни абстракции и е изцяло базиран върху стандартната библиотека net/http. Той използва стандартните типове http.Handler и http.HandlerFunc, без да въвежда собствени контекстни структури или обвивки. Това означава, че кодът остава преносим и може лесно да бъде използван с други HTTP библиотеки, без необходимост от пренаписване. Второ генерално предимство е възможността за композиция на middleware компоненти. chi използва стандартен функционален интерфейс, който позволява гъвкаво прилагане на middleware на различни нива – глобално, по групи от маршрути или за конкретни крайни точки. Това позволява ясно структуриране на логиката, като например логване, генериране на request идентификатори, обработка на грешки и CORS конфигурация. Допълнително, chi предоставя удобен механизъм за извличане на параметри от URL пътища чрез декларативен синтаксис. Това улеснява обработката на REST заявки, като позволява директен достъп до параметри като namespaced и име на ресурс.

В сравнение с алтернативи като Gin и Echo, chi не въвежда допълнителни абстракции като собствен контекст, което намалява сложността и зависимостите. Библиотеки като gorilla/mux са изключени поради липса на активна поддръжка, а стандартният net/http ServeMux не предоставя достатъчно функционалност за изграждане на пълноценен REST API.

MVC архитектура и слой за валидация

API компонентът е реализиран чрез строга Model-View-Controller (MVC) архитектура, при която разделението на отговорностите е ясно дефинирано и строго спазено. Контролерите са отговорни за обработката на HTTP заявките. Те декодират входящите данни, извличат параметри от URL, извършват базова (структурна) валидация и извикват съответните услуги. Те не взаимодействат директно с Kubernetes ресурси. Сервисният слой съдържа цялата логика за комуникация с Kubernetes API. Той приема обикновени Go структури като вход и връща резултати или типизирани грешки. В този слой не се използват HTTP специфични компоненти като `http.ResponseWriter`, което гарантира ясна изолация между бизнес логиката и транспортния слой. DTO (Data Transfer Object) слой дефинира структурата на данните, обменяни между API и потребителския интерфейс. Той включва функции за преобразуване между Kubernetes CRD обекти и формата, използван от API отговорите и бързо адаптиране на данните без директна зависимост от вътрешната структура на ресурсите.

Валидацията на данните е разделена на два основни типа.

- Структурната валидация се извършва на ниво контролер и гарантира, че входящите заявки са коректно форматирани и съдържат всички задължителни полета. При наличие на грешка се връща HTTP статус код 422 (Unprocessable Entity), което сигнализира за невалидни входни данни.
- Семантичната валидация се извършва от Kubernetes API сървър. Тя включва проверки за съществуващо състояние, конфликти или нарушения на дефинираните правила. Грешките от този тип се обработват в сервисния слой и се връщат като типизирани отговори.

Този двуслоен подход към валидацията осигурява ясно разграничение между синтактични и логически грешки, което подобрява надеждността и предвидимостта на системата.

6. Angular 21 и Signals API

Обосновка за избора на Angular

Компонентът `ɵɵngZone` представлява Single-Page Application (SPA), реализирано с Angular 21. Изборът на Angular пред алтернативни front-end технологии като React, Vue или Svelte е базиран на няколко архитектурни съображения. На първо място, Angular налага ясни структурни конвенции. Framework-ът дефинира разделение между компоненти (presentation слой), услуги (бизнес логика и HTTP комуникация) и модели (типове данни). Това съответства на слоестата архитектура на бекенд компонентите и намалява необходимостта от допълнителни архитектурни решения при разработка на нови функционалности. За разлика от Angular, React предоставя значителна гъвкавост, но не налага конкретна структура, което води до повишена когнитивна сложност при изграждане на специализирани системи като административни панели. Второ важно предимство е пълнотата на framework-a. Angular предоставя вградени механизми за HTTP комуникация (`HttpClient`), `dependency injection`, маршрутизация и работа с форми. Това елиминира необходимостта от интеграция на множество външни библиотеки, както е при React, където всяка функционалност (напр. HTTP заявки, state management, routing) изисква отделно решение. Angular е изцяло ориентиран към TypeScript. Всички API интерфейси, компоненти и услуги са строго типизирани, което осигурява крайна типова безопасност – от получаването на данни от API слоя до визуализацията им в шаблоните. Това намалява риска от runtime грешки и подобрява поддръжката на кода. В сравнение с други технологии, Angular предлага по-добра стандартизация и интеграция за изграждане на комплексни приложения. Vue предоставя по-лек модел, но с по-малко стандартизирани практики, а Svelte се отличава с висока производителност, но има ограничена екосистема за enterprise приложения.

Standalone компоненти – премахване на NgModule

В по-ранните версии на Angular всяка компонента трябваше да бъде декларирана в NgModule, който служеше за групиране на компоненти, управление на зависимости и конфигуриране на `dependency injection`. Това водеше до създаване на множество модули, често без директна връзка с конкретни функционалности. Angular 21 въвежда модела на Standalone компоненти, който премахва необходимостта от използване на NgModule. Всеки компонент е самостоятелна единица, която декларира своите зависимости директно чрез `imports`. Този подход предоставя няколко съществени предимства:

- зависимостите са ясно видими на ниво компонент;
- компонентите могат да бъдат зареждани динамично чрез router-a без допълнителни обвивки;
- оптимизацията на bundle-a (tree-shaking) е по-ефективна;
- тестването е опростено, тъй като не изисква конфигуриране на цели модули.

Реактивно управление на състоянието

Angular Signals представляват нов механизъм за реактивно управление на състоянието, въведен в Angular 16 и утвърден като основен подход в Angular 21. Традиционният модел за управление на състоянието в Angular използваше RxJS и

Observable/BehaviorSubject, което водеше до необходимост от управление на subscriptions и риск от memory leaks. Този подход изисква допълнителна логика за управление на жизнения цикъл на subscriptions и използване на async pipe в шаблоните. [13] Signals предоставят по-прост и директен модел:

```
tools = signal<Tool[]>([]);
loading = signal(true);

ngOnInit(): void {
    this.toolService.listTools().subscribe({
        next: tools => this.tools.set(tools),
        complete: () => this.loading.set(false),
    });
}
```

В този случай:

- не се изисква управление на subscriptions;
- няма риск от memory leaks;
- достъпът до данните в шаблона става чрез директно извикване (tools()).

Signals са интегрирани с механизма за промяна на състоянието в Angular, като осигуряват fine-grained реактивност. Това означава, че при промяна на даден signal се обновяват единствено зависимите части от шаблона, вместо да се извършва глобално обновяване на компонента, както при Zone.js. Допълнително, методът update() позволява лесна трансформация на текущото състояние:

```
this.tools.update(all => all.filter(t => t.name !== deletedName));
```

Това позволява реализиране на оптимистични UI обновления без повторно зареждане на данните.

Централизирана обработка на грешки

Angular предоставя механизъм за HTTP интерсептори, които позволяват обработка на всички HTTP заявки и отговори на едно централно място. В Eirenyx този механизъм се използва за унифицирана обработка на грешки от API слоя. Без използването на интерсептор, всяка компонента би трябвало самостоятелно да обработва HTTP грешките, което води до дублиране на логика и повишена сложност. Централизацията на тази функционалност осигурява:

- консистентно поведение при грешки;
- по-чист и поддържаем код;
- намаляване на дублирането.

Този подход е съвместим с модела на standalone компоненти и позволява по-ясна и декларативна конфигурация на приложението.

7. Kubernetes CRD модел като хранилище на състоянието

Защо не се използва външна база данни?

Архитектурата на Eirenyx не използва релационна база данни (напр. PostgreSQL) или документно-ориентирана база (напр. MongoDB) за съхранение на състоянието. Цялото състояние вече се съхранява в Kubernetes. Инструментите, управлявани от Eirenyx (Trivy, Falco, Litmus), съхраняват собственото си състояние като Kubernetes ресурси. Резултатите от Trivy сканиранията се съхраняват като VulnerabilityReport CRD обекти, а резултатите от Litmus експериментите – като ChaosResult CRD обекти. Не съществува външно състояние, което да изисква синхронизация – самият клъстер е единственият източник на истина (authoritative store). etcd осигурява силни гаранции за консистентност. Kubernetes API сървърът, базиран на etcd, предоставя механизми като оптимистичен контрол на конкурентния достъп, линейризуеми четения (linearizable reads) и атомарни операции за запис. Тези свойства са еквивалентни на гаранциите, предоставяни от релационните бази данни, но без необходимостта от поддръжка на отделен database клъстер. Добавянето на база данни въвежда допълнителна оперативна зависимост. Оператор, който разчита на PostgreSQL, не може да функционира при недостъпност на базата данни. Това води до т.нар. bootstrap проблем (инсталиране на оператор, който сам изисква външна зависимост), създава риск от единична точка на отказ (single point of failure) и увеличава оперативната сложност (архивиране, миграции, скалиране). Kubernetes API вече представлява REST интерфейс. Операторът, eirenyx-api и външните инструменти (kubectrl, GitOps контролери) могат да четат и записват ресурси чрез стандартния Kubernetes API. Това елиминира необходимостта от проектиране, имплементация и защита на отделен слой за достъп до база данни.

Трите CRD обекта и техните взаимоотношения

Eirenyx дефинира три типа CRD ресурси, които формират строга йерархия на собственост:

CRD	API група	Обхват	Обработка се от
Tool	eirenyx.io/v1alpha1	Namespace	ToolReconciler
Policy	eirenyx.io/v1alpha1	Namespace	PolicyReconciler
PolicyReport	eirenyx.io/v1alpha1	Namespace	PolicyReportReconciler

Йерархията се реализира чрез механизма на Kubernetes owner references. [1] Когато PolicyReconciler създава PolicyReport, той задава съответния Policy като негов собственик. Аналогично, когато Policy е асоциирана с даден Tool, Tool се задава като неин собственик.

Това води до следното поведение:

- Изтриването на Tool предизвиква каскадно изтриване на всички свързани Policy ресурси и съответните PolicyReport обекти.
- Изтриването на Policy води до изтриване на свързаните PolicyReport.

- Изтриването на PolicyReport няма ефект върху Policy, тъй като той ще бъде генериран отново при следващия цикъл на reconciliation.

Шаблон за „status subresource“

Всеки CRD използва status subresource, при който spec (желано състояние, задавано от потребителя) и status (наблюдавано състояние, управлявано от контролерите) са разделени в различни API endpoints с отделни права за достъп. Това разделение се прилага често в операторните системи, тъй като без него може да възникне следният сценарий:

1. Потребителят задава tool.spec.enabled = false.
2. Контролерът обработва промяната, изпълнява EnsureUninstalled и записва tool.status.installed = false.
3. Клиентът на потребителя, при повторен опит за запис, изпраща стара версия на обекта (status.installed = true).
4. Това презаписва актуалното състояние, записано от контролера.

С използването на status subresource:

- Потребителските заявки се изпращат към основния endpoint
- Контролерните записи към /status endpoint

Тези операции се проследяват с отделни версии на ресурса (resource version), което предотвратява конфликтно презаписване.

Интеграция с GitOps

Тъй като конфигурацията на Eirenyx се описва изцяло чрез Kubernetes YAML манифести, състоянието на сигурността на клъстера може да бъде управлявано чрез Git:

```
apiVersion: eirenyx.io/v1alpha1
kind: Tool
metadata:
  name: trivy
  namespace: security
spec:
  type: trivy
  enabled: true
```

GitOps контролери като Flux или ArgoCD автоматично прилагат тези конфигурации към клъстера при всяка промяна в Git хранилището. Този подход осигурява:

- централизирано управление на конфигурацията
- проследимост на всички промени (audit trail)
- контрол чрез pull request процеси
- синхронизация между код и инфраструктура

III. ПРАКТИЧЕСКА ИМПЛЕМЕНТАЦИЯ И АРХИТЕКТУРНО РАЗВИТИЕ НА СИСТЕМАТА

1. CRD-ориентирана архитектурна концепция

Основното архитектурно твърдение на системата Eirenuх е, че три съществено различни класа инструменти – инструмент за сканиране на уязвимости, система за откриване на заплахи при изпълнение и framework за chaos engineering. Те могат да бъдат управлявани чрез единен, последователен декларативен интерфейс, без да се компрометира конфигурационната гъвкавост на всеки отделен инструмент. Механизмът, който позволява реализацията на тази концепция, е използването на Custom Resource Definition (CRD). Kubernetes CRD предоставят възможност за дефиниране на произволни домейн-специфични обекти като пълноправни ресурси в API сървъра. След регистриране, тези ресурси се третират по същия начин като вградените обекти: съхраняват се в etcd, подлежат на RBAC контрол, могат да бъдат наблюдавани от контролери и управлявани чрез стандартни инструменти като kubectl.

Eirenuх използва тази разширяемост, за да дефинира собствена ресурсна терминология – Tool, Policy и PolicyReport, която абстрахира различните API модели на Trivy, Falco и LitmusChaos. В резултат се постига унифициран модел, при който платформеният инженер използва един и същ тип Kubernetes манифест за различни операции: стартиране на сканиране, конфигуриране на правила или изпълнение на експерименти. Така се осигурява единна повърхност за работа, еднакъв API, унифициран процес на deployment, стандартизирано отчитане на резултатите и централизирана audit следа. Единствената променлива част остава инструмент-специфичната конфигурация. В следващите секции се разглеждат трите CRD ресурса и операторният механизъм (reconcilers и factory pattern), чрез които декларативните заявки се преобразуват в реално поведение на инструментите.

Моделът на данните в Eirenuх е организиран като строго дефинирана трислойна йерархия:

```
Tool <- owns -> Policy <- owns -> PolicyReport
```

Всеки слой отговаря за различен аспект от управлението:

CRD	Отговорност	Създава се от
Tool	Декларира инсталация на инструмент	Платформен инженер
Policy	Декларира поведение на инструмента	Security инженер
PolicyReport	Съдържа резултатите от изпълнението	Оператор (автоматично)

Връзките между ресурсите се реализират чрез Kubernetes owner references, като така се налага автоматично почистване: при изтриване на Tool, Kubernetes премахва всички свързани Policy ресурси, а те от своя страна, всички PolicyReport обекти, без необходимост от допълнителна логика.

Tool CRD – декларация за инсталация

Ресурсът Tool представлява декларация на намерение: даден инструмент трябва да бъде инсталиран, в конкретен namespace и с определена конфигурация. Всеки Tool съответства на един Helm release.

```
apiVersion: eirenyx.io/v1alpha1
kind: Tool
metadata:
  name: trivy
  namespace: eirenyx-system
spec:
  type: trivy
  enabled: true
  namespace: trivy-system
  values:
    trivy:
      resources:
        requests:
          memory: "256Mi"
```

Полето spec.type е ограничено до три допустими стойности (trivy, falco, litmus) и служи като основен дискриминатор за избор на конкретна имплементация чрез factory pattern. Това ограничение се валидира от Kubernetes API сървъра чрез OpenAPI схема. Полето spec.enabled управлява оперативното състояние. Задаването му на false не изтрива ресурса, а задейства контролирано премахване на Helm release-a, което стартира временно деактивиране и последващо възстановяване без загуба на конфигурация. Полето spec.values позволява подаване на произволни Helm параметри чрез runtime.RawExtension, без необходимост от разширяване на CRD схемата.

Статусът на ресурса отразява реалното състояние:

```
type ToolStatus struct {
    Installed bool
    Healthy   bool
    Version   string
    Conditions []metav1.Condition
}
```

Installed указва наличието на Helm release, а Healthy – дали инструментът е в готово състояние. Механизмът Conditions позволява интеграция със стандартни Kubernetes инструменти като kubectl wait.

Към ресурса се прилагат finalizers (eirenyx.tool/finalizer), който гарантира че при изтриване ще бъде изпълнена необходимата логика за почистване.

Policy CRD – декларация за поведение

Ресурсът Policy дефинира какво трябва да извършва даден инструмент. Докато Tool описва инсталация, Policy описва поведение. Спецификацията използва дискриминиращо поле type и инструмент-специфични подструктури:

```
type PolicySpec struct {
    Type      PolicyType
    Enabled bool
    Target    PolicyTarget
    Falco     *FalcoPolicySpec
    Trivy     *TrivyPolicySpec
    Litmus    *LitmusPolicySpec
}
```

Само една от подструктурите може да бъде активна. Това се валидира от PolicyReconciler. Примерни политики:

```
# Trivy policy
spec:
  type: trivy
  trivy:
    namespace: production
    severity: CRITICAL,HIGH
```

```
# Falco policy
spec:
  type: falco
  falco:
    ruleRef: terminal-shell-in-container
```

```
# Litmus policy
spec:
  type: litmus
  litmus:
    experiments:
      - name: pod-delete-test
        experimentRef: pod-delete
```

Унифицираната структура позволява използване на един и същ модел за всички инструменти.

Статусът проследява жизнения цикъл:

```
type PolicyStatus struct {  
    Phase  
    Conditions  
    LastReport  
    ObservedGen  
}
```

ObservedGeneration позволява откриване на промени и гарантира, че резултатите са актуални.

PolicyReport CRD – резултатен модел

PolicyReport съдържа резултатите от изпълнението и се създава автоматично от оператора.

```
type PolicyReportSpec struct {  
    PolicyRef PolicyReference  
    Type      PolicyType  
}  
type PolicyReportStatus struct {  
    Phase  
    Summary  
    Details  
}
```

Phase описва състоянието (Pending, Running, Completed), а Summary съдържа агрегирани резултати:

```
type ReportSummary struct {  
    Verdict  
    TotalChecks  
    Passed  
    Failed  
}
```

Критериите за Verdict са инструмент-специфични:

- Trivy: наличие на критични уязвимости
- Falco: наличие на runtime събития
- Litmus: успешно изпълнение на експерименти

Полето Details съдържа детайлна информация във формат JSON, специфичен за инструмента.

Йерархия на собствеността и каскаден жизнен цикъл

Връзките от тип owner reference между трите CRD ресурса [1] не представляват само конвенция за именуване, а реализират цялостната политика за жизнения цикъл на системата. Когато даден ресурс от тип Tool бъде изтрит, се изпълнява следната последователност от действия:

- Kubernetes маркира всички Policy ресурси, които съдържат ownerReference – Tool, за изтриване.
- Изтриването на всеки Policy ресурс води до маркиране за изтриване на съответния PolicyReport.
- Finalizer-ът на ToolReconciler изпълнява EnsureUninstalled, преди ресурсът Tool да бъде премахнат от etcd.
- Finalizer-ът на PolicyReconciler изпълнява Cleanup върху съответния engine, като премахва специфичните за инструмента ресурси (Job, ConfigMap, ChaosEngine).
- След като всички зависими обекти бъдат премахнати, се изтрива и родителският ресурс.

Този каскаден модел елиминира цял клас от течове на ресурси. При отсъствие на owner references, изтриването на Tool би могло да остави след себе си осиротели Policy обекти, които сочат към несъществуваща инсталация, а тези политики от своя страна биха могли да оставят остарели PolicyReport ресурси с резултати, които вече не съответстват на реалното състояние на клъстера. Чрез йерархията на собствеността този риск е елиминиран на архитектурно ниво.

В Eirenuх връзката Policy – Tool се установява отложено (lazy) при първото изпълнение на reconciliation логиката:

```
has, err := controllerutil.HasOwnerReference(policy.OwnerReferences, &tool,
r.Scheme)
if !has {
    controllerutil.SetOwnerReference(&tool, &policy, r.Scheme)
    r.Update(ctx, &policy)
    return Complete()
}
```

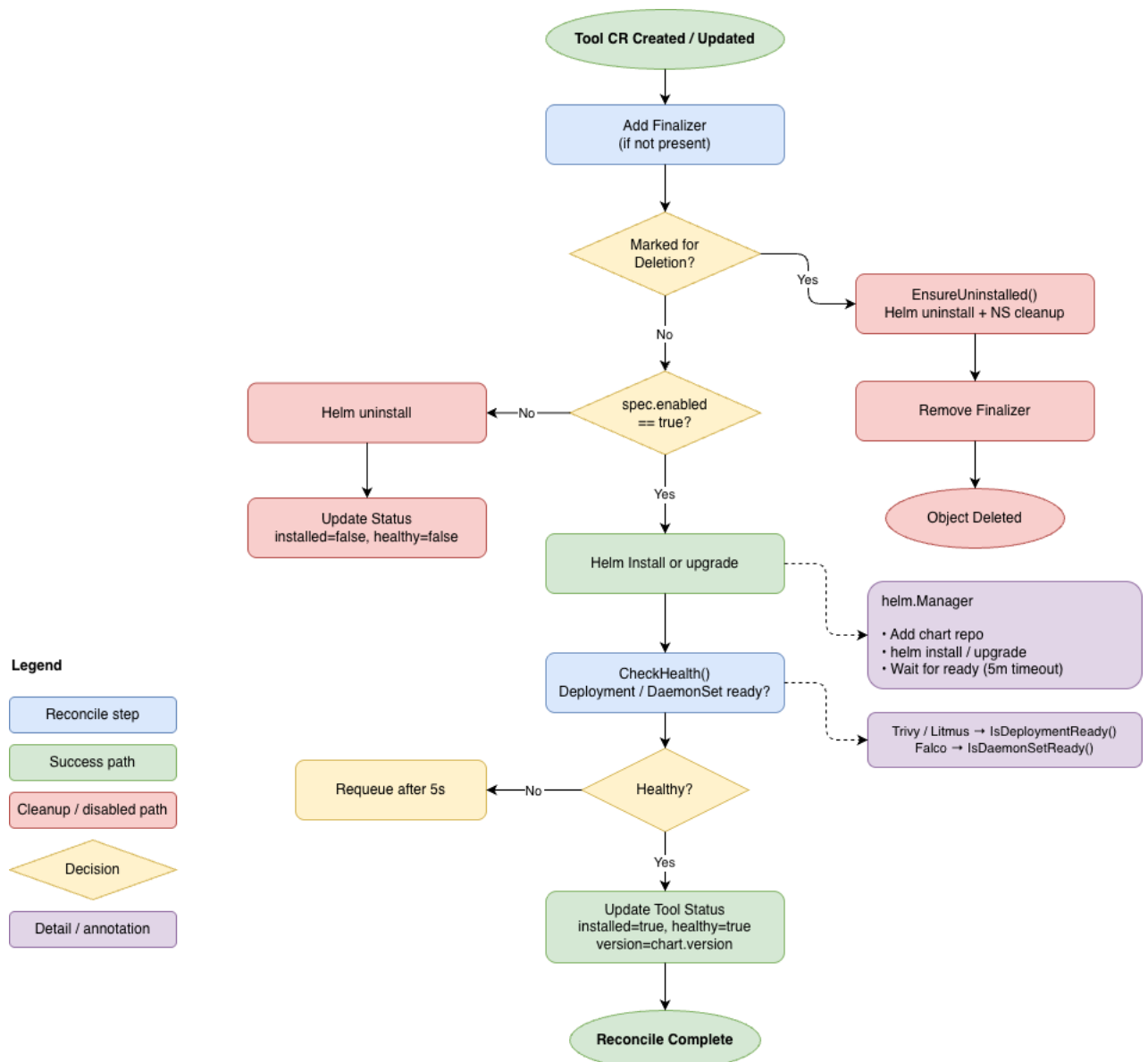
Този отложен механизъм е необходим, тъй като ресурсите Tool и Policy могат да бъдат създавани независимо един от друг, например при прилагане на GitOps хранилище, в което манифестите се обработват по азбучен ред. Поради това операторът не изисква Tool ресурсът да съществува преди създаването на съответната Policy; вместо това той изчаква и повтаря опита, докато зависимият ресурс стане наличен.

2. Процес на реконсиляция на CRD (Tools, Policy, PolicyReport)

Като основна входна точка на системата, може да се започне през ToolReconciler, който представлява основният операторен цикъл за CRD ресурса Tool. Неговият метод Reconcile се извиква от controller-runtime при създаване, промяна, изтриване или повторно планирано изпълнение на Tool ресурс. Неговата задача е да синхронизира жизнения цикъл на Helm инсталацията с желаното състояние, дефинирано в spec. Логиката на reconciler-а следва ясно дефинирана седемстъпкова последователност.

- Стъпка 1 – Извличане на ресурса: Reconciler-ът извлича Tool обекта чрез неговия NamespacedName. Ако обектът вече не съществува (например е бил изтрит преди изпълнението на reconciliation), грешката се игнорира чрез client.IgnoreNotFound, а изпълнението приключва.
- Стъпка 2 – Обработка при изтриване: Ако полето DeletionTimestamp е зададено, това означава, че ресурсът е в процес на изтриване. В този случай reconciler-ът извиква EnsureUninstalled, а при успешно завършване премахва finalizer-а. Така се гарантира, че Helm release-ът винаги ще бъде премахнат, преди Tool ресурсът да бъде окончателно изтрит от etcd.
- Стъпка 3 – Регистрация на finalizer: Ако finalizer липсва, той се добавя чрез patch операция, а не чрез пълен update. Това решение предотвратява неволно презаписване на конкурентни промени, например върху status подресурса.
- Стъпка 4 – Инстанциране на сервис: Извиква се factory функцията NewToolService, която на база spec.type връща правилната имплементация на ToolService (TrivyService, FalcoService или LitmusService). Ако типът не е разпознат, reconciler-ът записва условие Ready=False и прекратява изпълнението.
- Стъпка 5 – Инсталиране или деинсталиране: Логиката се разклонява според стойността на spec.enabled. Ако тя е true, методът EnsureInstalled изпълнява Helm инсталация или обновяване и изчаква до пет минути ресурсът да достигне готово състояние. Ако стойността е false, EnsureUninstalled премахва Helm release-а, ако такъв съществува.
- Стъпка 6 – Проверка на състоянието: Методът CheckHealth проверява състоянието на реално инсталирания workload. За Trivy и Litmus се проверява броят на готовите реплики в Deployment, а за Falco – стойността numberReady спрямо desiredNumberScheduled в DaemonSet. Ако инструментът все още не е готов, reconciler-ът се планира за повторно изпълнение след пет секунди.
- Стъпка 7 – Обновяване на status: Наблюдаваното състояние (installed, healthy) се записва в status подресурса. Ако инструментът все още не е здравословен, се изпълнява ново queue действие, така че контролерът да не преминава в неактивно състояние, докато workload-ът стартира.

Помощните функции Complete(), CompleteWithError() и RequeueAfter() представляват тънки обвивки около ctrl.Result, които позволяват бизнес логиката да остане независима от шаблонния framework код. Цялостният жизнен цикъл на ToolReconciler е илюстриран на Фигура 4, където е показан потокът на инсталация, проверка на състоянието и контролирано премахване на управлявания инструмент, докато в Приложение 1а е приложен изгодният код за процесът на реконсиляция.



Фигура 4: Поток на жизнения цикъл в Tool Reconciler

На свой ред PolicyReconciler координира едновременно три основни аспекта: наличността на инструмента, изпълнението на политиката и генерирането на отчет. Той се активира при всяка промяна в ресурс от тип Policy. Първото действие на PolicyReconciler е да открие Tool ресурса, към който принадлежи дадената политика. Търсенето използва конвенция за именуване: политика с spec.type: trivy очаква да съществува Tool с име trivy в същия namespace. Ако инструментът не бъде намерен или не е в готово състояние, reconciler-ът отлага изпълнението и изчаква, тъй като политика не може да бъде приложена върху неинсталиран инструмент.

След установяване на собствеността и при условие, че политиката е активирана, изпълнението се делегира към policy.Engine, върнат от factory механизма:


```
engine, err := NewPolicyEngine(&policy, deps)
engine.Validate(&policy)
engine.Reconcile(ctx, &policy)
```

Методът `Validate` извършва проверка за структурна коректност, преди да бъде модифицирано състоянието на клъстера. Грешките на този етап се третират като крайни, тъй като представляват потребителски грешки и не могат да бъдат решени чрез повторен опит.

Методът `Reconcile` изпълнява конкретното действие за съответния инструмент – създаване на Kubernetes Job за Trivy, запис на ConfigMap за Falco или създаване на ChaosEngine CRD за Litmus.

След изпълнение на политиката `reconciler`-ът извиква `engine.GenerateReport`, който създава или нулира `PolicyReport` ресурса. Ако `generation` стойността на политиката се е увеличила, това означава, че конфигурацията е била променена и старият отчет вече не е валиден. В този случай `PolicyReport` се връща в състояние `Pending`, което принуждава `PolicyReportReconciler` да изпълни нов анализ и да генерира актуален резултат. Без този механизъм остарели отчети биха могли да останат маркирани като `Completed`, въпреки че вече не съответстват на текущата конфигурация. В Приложение 1б е представен кодът за реконсиляция на `Policy`.

`PolicyReportReconciler` е специализиран контролер с ясно ограничен обхват: да придвижи ресурс от тип `PolicyReport` от състояние `Pending` или `Running` към `Completed`. Той е реализиран отделно от `PolicyReconciler`, за да бъде спазен принципът за единствена отговорност – контролерът за политики управлява жизнения цикъл и задейства изпълнението, докато контролерът за отчети извършва анализа и записва резултатите.

Две защитни проверки предотвратяват ненужно повторно изпълнение:

```
// Already done – do not re-run
if policyReport.Status.Phase == eirenyx.ReportCompleted {
    return Complete()
}

// Owner is gone – clean up the orphan
if apierrors.IsNotFound(err) {
    r.Delete(ctx, &policyReport)
    return Complete()
}
```

Изпълнението се делегира към `report.Handler`, върнат от `factory` механизма:

```
handler, err := NewReportEngine(&policyReport, deps)
handler.Reconcile(ctx, &policyReport)
```

Интерфейсът Handler е умишлено опростен и съдържа единствено метод Reconcile. Конкретните имплементации могат да извършват различен брой Kubernetes API операции, да изчакват завършване на Job, да четат инструмент-специфични CRD ресурси или да обобщават резултати. Самият контролер остава напълно независим от тази сложност – той не знае дали работи с Trivy VulnerabilityReport, Falco alerts или Litmus ChaosResult.

Factory pattern и основните интерфейси

Factory pattern е механизмът, който позволява трите еднотипни reconciler компонента да управляват три съществено различни инструмента. Самите контролери не съдържат никаква условна логика, зависеща от типа на инструмента. Цялото разклоняване е концентрирано в три factory функции, разположени в `internal/factory/factory.go`.

```
type Dependencies struct {
    Client      client.Client
    Scheme      *runtime.Scheme
    K8sClient   *k8s.Client
}

func NewToolService(tool *eirenyx.Tool, deps Dependencies)
(tool.ToolService, error)

func NewPolicyEngine(p *eirenyx.Policy, deps Dependencies) (policy.Engine,
error)

func NewReportEngine(r *eirenyx.PolicyReport, deps Dependencies)
(report.Handler, error)
```

Структурата Dependencies изпълнява ролята на лек контейнер за dependency injection. Тя съдържа `client.Client` за достъп до Kubernetes API, `Scheme` за работа с owner references и регистрация на типове, както и ниско ниво `k8s.Client`. Предаването на зависимостите по експлицитен начин, вместо използване на глобално състояние, прави factory функциите лесни за тестване и повторно използване. Условната логика за избор на конкретна имплементация е концентрирана единствено във factory функциите:

```
func NewToolService(tool *eirenyx.Tool, deps Dependencies)
(tool.ToolService, error) {
    switch tool.Spec.Type {
    case eirenyx.ToolFalco:
        return &tools.FalcoService{K8sClient: deps.K8sClient}, nil
    case eirenyx.ToolLitmus:
        return &tools.LitmusService{K8sClient: deps.K8sClient}, nil
    case eirenyx.ToolTrivy:
        return &tools.TrivyService{K8sClient: deps.K8sClient}, nil
    default:
```

```

        return nil, fmt.Errorf("unsupported tool type: %s", tool.Spec.Type)
    }
}

```

Архитектурното обосновка е както следва – добавянето на нов инструмент изисква единствено:

- дефиниране на нова константа в `api/v1alpha1`;
- реализиране на съответните структури (`Service`, `Engine`, `Handler`);
- добавяне на нов `case` във `factory` функциите.

Не се налага промяна в кода на `reconciler`-ите. Така системата реализира пряко принципа `Open/Closed` – отворена за разширение и затворена за модификация.

```

type ToolService interface {
    Name() string
    EnsureInstalled(ctx context.Context, tool *eirenyx.Tool) error
    EnsureUninstalled(ctx context.Context, tool *eirenyx.Tool) error
    CheckHealth(ctx context.Context, tool *eirenyx.Tool) (bool, error)
}

```

`ToolService` дефинира договор за управление на жизнения цикъл на инсталацията. Всички методи работят с `Tool CRD` и имат достъп до неговата конфигурация, включително `Helm` параметрите. Методите `EnsureInstalled` и `EnsureUninstalled` са идемпотентни по договор, което е критично за операторна логика, където една и съща `reconciliation` функция може да бъде извиквана многократно. `Engine` дефинира договор за изпълнение на политики. Методите отразяват отделни фази от жизнения цикъл:

- `Validate`: структурна проверка на конфигурацията;
- `Reconcile`: създаване или актуализиране на нужните `Kubernetes` ресурси;
- `Cleanup`: премахване на тези ресурси при деактивиране или изтриване;
- `GenerateReport`: създаване на `PolicyReport` в начално състояние.

```

type Engine interface {
    Validate(policy *eirenyx.Policy) error
    Reconcile(ctx context.Context, policy *eirenyx.Policy) error
    Cleanup(ctx context.Context, policy *eirenyx.Policy) error
    GenerateReport(ctx context.Context, policy *eirenyx.Policy)
    (*eirenyx.PolicyReport, error)
}

```

Handler е най-простият от трите интерфейса. Единственият му метод придвижва PolicyReport към крайно състояние. Разделението между Engine и Handler е умишлено — стартирането на изпълнение и обработката на резултатите имат различна времева динамика и различни изисквания към състоянието.

```
type Handler interface {  
    Reconcile(ctx context.Context, policyReport *eirenyx.PolicyReport)  
    error  
}
```

Комбинацията между factory pattern и трислойния CRD модел формира архитектура, при която сложността на три различни инструмента е напълно абстрахирана зад единен Kubernetes API. Взаимодействието със системата се осъществява чрез последователни, но логически разграничени етапи, които следват една и съща структурна логика независимо от конкретния инструмент.

Процесът може да бъде обобщен в три основни фази:

- Деклариране и управление на инструмент (Tool lifecycle):
 - Потребителят създава ресурс от тип Tool, чрез който декларира желанието даден инструмент да бъде инсталиран и управляван.
 - ToolReconciler обработва този ресурс, като чрез factory функция избира съответната имплементация (TrivyService, FalcoService или LitmusService).
 - След това, чрез Helm SDK се извършва инсталация или обновяване на съответния инструмент в клъстера (напр. DaemonSet за Falco, Deployment за Trivy operator, Litmus operator).
- Деклариране и изпълнение на политика (Policy execution):
 - Потребителят създава ресурс от тип Policy, който дефинира конкретното поведение на вече инсталирания инструмент.
 - PolicyReconciler използва factory механизъм, за да избере съответния Engine (Trivy, Falco или Litmus).
 - В зависимост от типа на политиката се създават необходимите Kubernetes ресурси:
 - Job за стартиране на Trivy сканиране
 - ConfigMap за конфигуриране на Falco правила
 - ChaosEngine CRD за изпълнение на Litmus експерименти
- Генериране и обработка на резултати (Report lifecycle):
 - След стартиране на изпълнението се създава ресурс PolicyReport, който отразява резултатите.
 - PolicyReportReconciler избира съответния Handler чрез factory функция и извършва събирането и обработката на резултатите.
 - В резултат на това статусът на PolicyReport се попълва с агрегирана информация, включително verdict (Pass/Fail) и детайлни данни за откритите събития или уязвимости.

Извод от използването на този модел е, че на ниво контролери, логиката остава напълно унифицирана. Всички ресурси преминават през една и съща последователност от операции – извличане, валидиране, изпълнение и отчитане. Различията между отделните инструменти са изолирани единствено в конкретните имплементации на

Service, Engine и Handler. За платформения инженер тази вътрешна вариативност остава напълно скрита. Независимо дали се управлява Trivy, Falco или Litmus, взаимодействието със системата се осъществява чрез един и същ декларативен модел – чрез Kubernetes манифести или чрез потребителския интерфейс на Eirenyx. Това осигурява последователност, намалява сложността на управление и минимизира риска от грешки при работа с различни инструменти.

3. Интеграция на Trivy

Trivy представлява цялостен open-source инструмент за сканиране на уязвимости и неправилни конфигурации, разработен от Aqua Security. В контекста на Eirenyx, Trivy изпълнява ролята на компонент за сигурност преди изпълнение (pre-runtime security) – анализира контейнерни образи за известни уязвимости (CVE) преди или по време на внедряване и предоставя оценка (verdict) дали зависимостите на даден workload съдържат критични рискове. Оперативният проблем, който Trivy решава в рамките на Eirenyx, е съществен. В Kubernetes среда с множество услуги, контейнерните образи могат да бъдат изтегляни от различни регистри, с различни версии и базови образи. При липса на централизирана политика за сканиране, отговорността остава на отделните екипи – те трябва ръчно да изпълняват сканиране, да анализират резултатите и да предприемат действия. [8,9] На практика това често не се случва систематично. Eirenyx автоматизира този процес, като:

- дефинира сканирането чрез Kubernetes ресурси;
- изпълнява го като задачи (Jobs) в клъстера;
- предоставя резултатите като структурирани и достъпни PolicyReport ресурси.

Важно е да се подчертае, че Trivy не се имплементира наново в Eirenyx. Той се инсталира чрез Helm chart (trivy-operator) и функционира като компонент в клъстера. Eirenyx добавя слой за управление – инициира сканирания, следи състоянието им и агрегира резултатите.

При създаване и активиране на ресурс Tool от тип Trivy, ToolReconciler инсталира trivy-operator чрез Helm в съответното namespace (обикновено trivy-system). Този оператор наблюдава определени ресурси (в случая Job) и генерира VulnerabilityReport CRD обекти като резултат. Процесът на интеграция и изпълнение на Trivy в рамките на Eirenyx може да бъде описан като последователна верига от взаимосвързани действия, реализирани чрез Kubernetes контролери и ресурси. Първоначално се създава ресурс от тип Policy със зададен тип trivy, който дефинира намерението за извършване на сканиране. Този ресурс се обработва от ToolReconciler, който гарантира, че необходимият инструмент (trivy-operator) е инсталиран в клъстера чрез Helm, като това осигурява изпълнителната среда за последващите операции. След като инструментът е наличен, PolicyReconciler поема обработката на политиката. Той използва trivy.Engine, за да трансформира декларативното описание на сканирането в конкретен Kubernetes ресурс – Job, който стартира процес по анализ на зададения контейнерен образ. Създаденият Job се засича автоматично от trivy-operator, който изпълнява самото сканиране. След приключване на анализа, резултатите се записват в специализиран ресурс от тип VulnerabilityReport, който съдържа структурирана информация за откритите уязвимости. На последния етап, PolicyReportReconciler извлича тези резултати чрез TrivyReportHandler. Той обработва и агрегира данните от VulnerabilityReport, след което ги трансформира в унифициран формат и ги записва в

ресурс от тип PolicyReport. Този ресурс представлява крайният резултат от политиката и предоставя обобщена оценка (verdict), както и детайлна информация за установените рискове.

Всеки етап се управлява от различен контролер, което позволява асинхронна обработка. PolicyReconciler създава задачата и не изчаква нейното изпълнение. Резултатите се обработват впоследствие от PolicyReportReconciler.

Сканирането с Trivy се дефинира чрез полето spec.trivy в ресурса Policy. То съдържа списък от сканирания, всяко от които е насочено към конкретен контейнерен образ:

```
apiVersion: eirenyx.io/v1alpha1
kind: Policy
metadata:
  name: api-image-scan
  namespace: eirenyx-system
spec:
  type: trivy
  enabled: true
  trivy:
    scans:
      - name: api
        image: myregistry/my-api:v1.2.3
        severity: "CRITICAL,HIGH"
        ignoreUnfixed: true
      - name: sidecar
        image: myregistry/my-sidecar:v2.0.0
        severity: "CRITICAL"
```

Всеки запис съдържа:

Поле	Предназначение
name	Уникален идентификатор на сканирането
image	Контейнерният образ за анализ
severity	Нива на уязвимости (еквивалент на --severity)
ignoreUnfixed	Игнориране на уязвимости без наличен fix

Валидацията (Validate) гарантира, че:

- съществува поне едно сканиране;
- всяко има валидни name и image.

Методът `trivy.Engine.Reconcile` създава по един `Job` за всяко сканиране. Името на задачата се генерира детерминистично:

```
func getScanJobName(policy *eirenyx.Policy, scanName string) string {
    return fmt.Sprintf("eirenyx-trivy-%s-%s", policy.Name, scanName)
}
```

Това гарантира идемпотентност, ако задачата вече съществува, не се създава нова. Конфигурацията на задачата е следната:

```
&batchv1.Job{
    Spec: batchv1.JobSpec{
        BackoffLimit:          ptr(int32(0)),
        TTLSecondsAfterFinished: ptr(int32(1800)),
        Template: corev1.PodTemplateSpec{
            Spec: corev1.PodSpec{
                RestartPolicy: corev1.RestartPolicyNever,
                Containers: []corev1.Container{{
                    Name:      "trivy",
                    Image:    "aquasec/trivy:latest",
                    Args:      buildTrivyArgs(scan),
                }},
            },
        },
    },
}
```

Основни решения:

- Без `retry` (`BackoffLimit: 0`)
- Автоматично почистване (`TTL: 30 минути`)
- Еднократно изпълнение (`RestartPolicy: Never`)
- `OwnerReference` към `Policy`

Аргументите се генерират динамично:

```
func buildTrivyArgs(scan eirenyx.TrivyScan) []string {
    args := []string{"image", "--format", "json", "--quiet"}
    if scan.Severity != "" {
        args = append(args, "--severity", scan.Severity)
    }
    if scan.IgnoreUnfixed {
        args = append(args, "--ignore-unfixed")
    }
}
```

```
args = append(args, scan.Image)
return args
}
```

Еквивалентна CLI команда:

```
trivy image --format json --quiet --severity CRITICAL,HIGH --ignore-unfixed
myregistry/my-api:v1.2.3
```

TrivyReportHandler обработва резултатите от сканирането. Той съществува отделно, тъй като сканирането е асинхронен процес.

Основни стъпки:

1. Извличане на VulnerabilityReport ресурси
2. Корелация по име на Job
3. Изчакване при липса на резултат
4. Агрегиране на данни
5. Изчисляване на verdict
6. Запис в PolicyReport

Пример за агрегиране:

```
total += int32(summary.CriticalCount + summary.HighCount + ...)
failed += int32(summary.CriticalCount + summary.HighCount)
```

Правило за verdict:

- CRITICAL или HIGH => Fail
- останалите => Pass

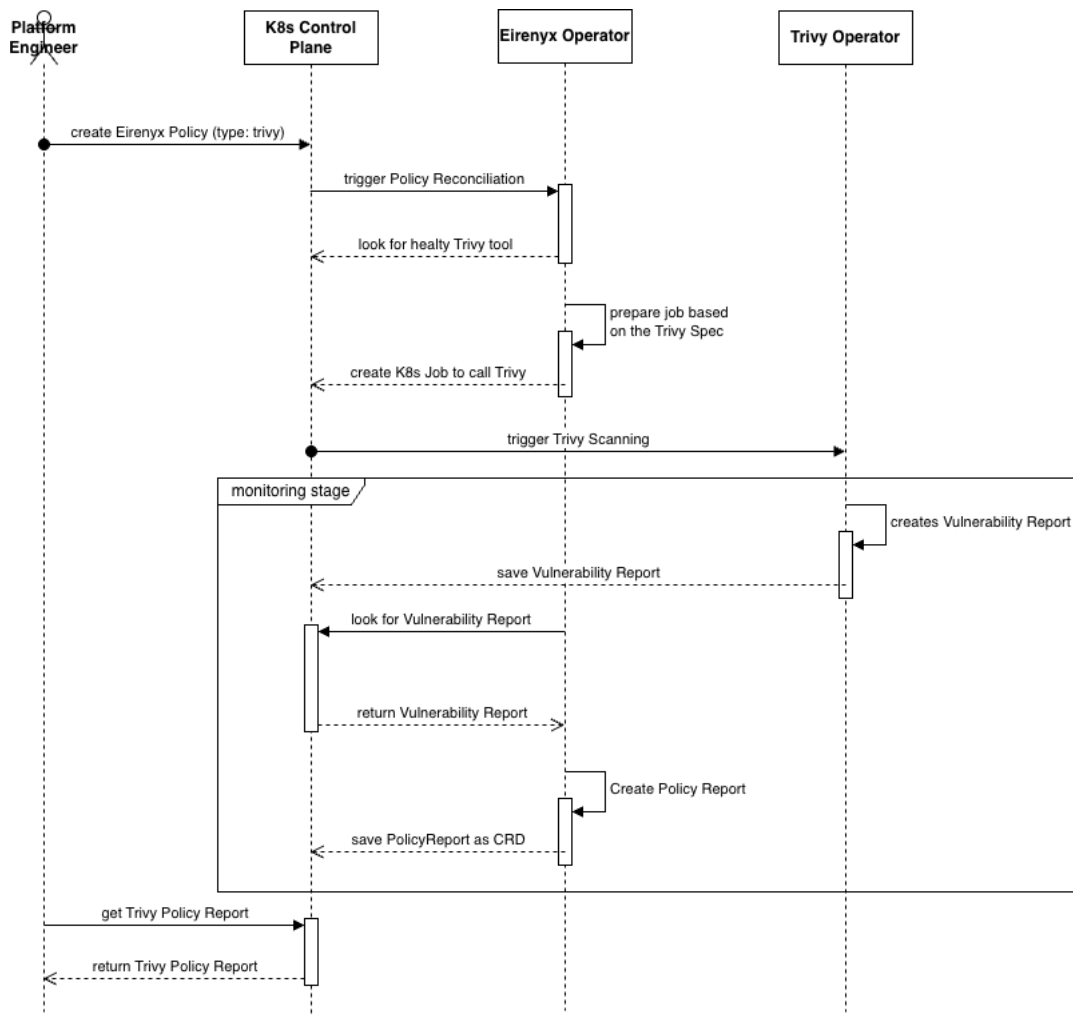
Запис:

```
policyReport.Status.Summary = eirenyx.ReportSummary{
    Verdict: verdict,
    TotalChecks: total,
    Passed: total - failed,
    Failed: failed,
}
```

Разделението между Engine и ReportHandler отразява реалния жизнен цикъл на процеса по сканиране и осигурява ясно разграничение между инициирането на проверката, нейното изпълнение и последващата обработка на резултатите. Engine компонентът отговаря за стартиране на сканирането чрез създаване на Kubernetes Job, който изпълнява конкретната проверка върху зададения артефакт или workload. След приключване на задачата, ReportHandler поема отговорността за наблюдение и интерпретация на генерираните резултати, като ги агрегира и записва в съответния PolicyReport ресурс. Този архитектурен подход повишава устойчивостта на системата

при рестарт на контролера, тъй като състоянието на изпълнението се съхранява в Kubernetes ресурси, а не в локална памет. Освен това се избягва повторно стартиране на вече изпълнени сканирания, като съществуващите задачи и отчети могат да бъдат разпознати при следващ reconciliation цикъл. В резултат се осигурява постоянен и проследим запис на резултатите, който може да бъде използван за одит, анализ на сигурността и последваща оценка на състоянието на системата.

PolicyReport функционира като исторически запис на състоянието на сигурността към даден момент. Фигура 5 илюстрира този процес и потока на данните между компонентите.



Фигура 5: Последователностна диаграма на Trivy реконсиляция

4. Интеграция на Falco

Falco представлява cloud-native инструмент за сигурност по време на изпълнение (runtime security), разработен под егидата на Cloud Native Computing Foundation (CNCF). Той открива аномално и потенциално злонамерено поведение в изпълняващи се контейнери, като се интегрира директно с Linux ядрото – чрез eBPF или kernel module и анализира потока от системни повиквания (syscalls), генерирани от всички процеси в клъстера. Докато Trivy отговаря на въпроса „съдържа ли този контейнерен образ известни уязвимости?“, Falco адресира различен аспект на сигурността: „наблюдава ли се подозрително поведение в момента?“. Двата инструмента се допълват взаимно. Един контейнер може да няма известни CVE уязвимости, но въпреки това да бъде компрометиран по време на изпълнение, например чрез изтегляне и изпълнение на злонамерен бинарен файл, стартиране на shell процес или достъп до чувствителни файлове. В рамките на Eirenyx, Falco изпълнява ролята на компонент за runtime сигурност, осигурявайки непрекъснат мониторинг на изпълняващите се workload-и. Заедно с Trivy се постига многослоен модел на защита (defence in depth), обхващащ както статичния, така и динамичния аспект на сигурността. [10]

Интеграцията на Falco в Eirenyx решава и проблема с оперативната фрагментация. В стандартна среда, конфигурирането на Falco изисква ръчно редактиране на ConfigMap-и или rule файлове, рестартиране на DaemonSet и отделен анализ на генерираните аларми. Eirenyx абстрахира този процес чрез Policy и PolicyReport CRD ресурси, предоставяйки унифициран декларативен интерфейс. При създаване и активиране на Tool ресурс от тип Falco, ToolReconciler инсталира Falco чрез Helm chart. Falco се изпълнява като DaemonSet, тъй като трябва да наблюдава системните повиквания на всяка нода поотделно. DaemonSet-ът работи с повишени привилегии (достъп до kernel или eBPF), което представлява контролиран компромис в сигурността. В продукционни среди това се управлява чрез RBAC и Pod Security политики. [1] След стартиране, Falco непрекъснато оценява своите правила спрямо системните събития. Тези правила са дефинирани в YAML DSL и могат да бъдат зареждани от множество източници. В Eirenyx те се подават чрез ConfigMap ресурси.

Falco политиката дефинира кои правила да бъдат наблюдавани. Поддържат се два взаимно изключващи се подхода:

- ruleRef: избор на конкретно правило по име
- ruleSelector: избор по тагове и приоритет

```
# Политика за конкретно правило
apiVersion: eirenyx.io/v1alpha1
kind: Policy
metadata:
  name: shell-detection
  namespace: eirenyx-system
spec:
  type: falco
  enabled: true
  falco:
    observe:
      ruleRef:
        name: "Terminal shell in container"

# Политика по тагове
apiVersion: eirenyx.io/v1alpha1
```

```

kind: Policy
metadata:
  name: network-monitoring
  namespace: eirenyx-system
spec:
  type: falco
  enabled: true
  falco:
    observe:
      ruleSelector:
        tags: [ "network" ]
        minPriority: "WARNING"

```

Валидацията гарантира, че:

- е зададен точно един от двата подхода;
- няма двусмислени конфигурации.

За разлика от Trivy, Falco не използва batch задачи, а персистентна конфигурация чрез ConfigMap.

```

func (e *Engine) Reconcile(ctx context.Context, policy *eirenyx.Policy)
error {
    cm := &corev1.ConfigMap{
        ObjectMeta: metav1.ObjectMeta{
            Name:      falcoPolicyConfigMapName(policy),
            Namespace: policy.Namespace,
        },
    }

    _, err := controllerutil.CreateOrUpdate(ctx, e.Client, cm, func() error {
        specBytes, _ := json.Marshal(policy.Spec.Falco)
        cm.Data["falcoPolicy.json"] = string(specBytes)
        return controllerutil.SetControllerReference(policy, cm, e.Scheme)
    })
    return err
}

```

Основни характеристики:

- идемпотентност чрез CreateOrUpdate
- автоматично почистване чрез OwnerReference
- динамично обновяване на правилата

Falco автоматично презарежда правилата при промяна на ConfigMap, без рестарт на DaemonSet.

Falco не генерира CRD ресурси като Trivy. Вместо това излъчва събития. Затова FalcoReportHandler работи чрез броене на събития.

```

eventCount := h.getReportEventOccurrence(ctx, ruleName)

verdict := eirenyx.VerdictPass
if eventCount > 0 {

```

```
    verdict = eirenyx.VerdictFail
}
```

Логиката е:

- 0 събития => Pass
- ≥ 1 събитие => Fail

Този строг подход е оправдан, тъй като Falco засича аномалии, които не трябва да се случват.

Falco отчетите се допълват с информация за подове:

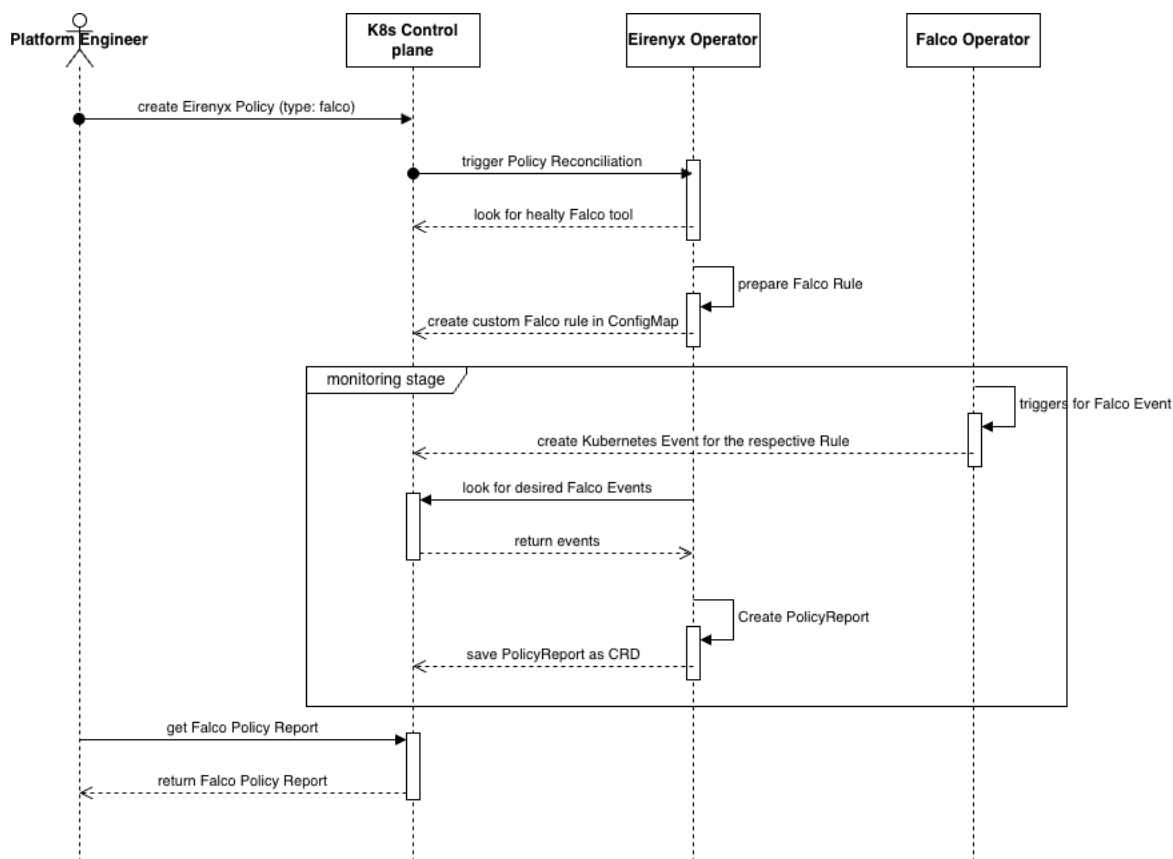
```
podDetails, _ := report.GetPodDetails(ctx, h.Client)

details := FalcoReportDetails{
    Message:    fmt.Sprintf("Detected %d occurrence(s)", eventCount),
    RuleName:   ruleName,
    PodDetails: podDetails,
}
```

Така се установява директна връзка между събития и workload-и.

Както при интеграцията с Trivy. При Falco имаме асинхронно изчакване, и polling на ресурси. Отчетите се генерират незабавно при появата на event генериран от Falco контролера.

Falco отчетите представят моментно състояние на сигурността, а не snapshot от сканиране. Механизмът с generation гарантира актуалност – всяка промяна в политиката води до нов отчет. С течение на времето, поредица от PolicyReport ресурси описва поведението на системата и наличието на аномалии. Фигура 6 илюстрира пълния поток на Falco политиката – от декларация, през конфигурация, до генериране на отчет.



Фигура 6: Последователна диаграма на Falco реконсилация

5. Интеграция на Litmus

LitmusChaos представлява Kubernetes-native framework за chaos engineering, който валидира устойчивостта на приложенията чрез умишлено инжектиране на контролирани откази и анализ на реакцията на системата. Той отговаря на въпрос, който нито Trivy, нито Falco могат да покрият: „продължава ли услугата да функционира коректно при възникване на откази?“. Принципите на chaos engineering, въведени първоначално от Netflix, предполагат, че разпределените системи неизбежно ще бъдат подложени на различни типове откази – мрежови проблеми, сринове на възли, изчерпване на ресурси. Вместо тези слабости да се откриват в продукционна среда, те следва да бъдат идентифицирани предварително чрез контролирани експерименти. LitmusChaos реализира този подход в Kubernetes чрез библиотека от предварително дефинирани експерименти – изтриване на подове, CPU натоварване, мрежова латентност, запълване на дисково пространство и други. [11]

В рамките на Eirenyx, Litmus изпълнява ролята на компонент за валидиране на устойчивостта (resilience validation), допълвайки Trivy (pre-deployment сигурност) и Falco (runtime наблюдение). Така се изгражда трислоен модел на защита, обхващащ както сигурността, така и надеждността на системата. Без Eirenyx, използването на Litmus изисква директна работа с ChaosEngine и ChaosResult CRD ресурси, както и задълбочено разбиране на вътрешния модел на Litmus. Eirenyx абстрахира тази сложност чрез унифицираните ресурси Policy и PolicyReport. При създаване и активиране на Tool ресурс от тип Litmus, ToolReconciler инсталира LitmusChaos чрез

Helm chart. Това води до деплойване на контролер (chaos-operator), който наблюдава ChaosEngine ресурси и изпълнява съответните експерименти.

Интеграцията на Litmus в Eirenyx се реализира като последователен процес, при който декларативно дефинирана политика се преобразува в конкретни chaos експерименти, изпълнявани в Kubernetes клъстера. Процесът започва със създаване на Policy ресурс от тип litmus, който описва какви експерименти следва да бъдат приложени спрямо определен workload. След това ToolReconciler гарантира, че LitmusChaos е наличен в клъстера, като при необходимост инсталира litmus-operator чрез Helm. След като инструментът е инсталиран, PolicyReconciler обработва конкретната политика чрез litmus.Engine. Неговата задача е да трансформира всяка дефиниция на експеримент в отделен ChaosEngine ресурс. Всеки такъв ресурс представлява декларация за изпълнение на конкретен chaos сценарий спрямо определен целеви workload. Когато litmus-operator засече създаден ChaosEngine, той автоматично създава т.нар. *chaos runner pod*. Този pod изпълнява съответния експеримент върху целевото приложение, например чрез изтриване на pod, натоварване на процесора или въвеждане на мрежова латентност. След приключване на експеримента Litmus записва резултата в ресурс от тип ChaosResult. Този ресурс съдържа информация за изпълнението и служи като източник на данни за следващия етап от обработката. На последния етап PolicyReportReconciler, чрез LitmusReportHandler, обработва наличната информация и я преобразува в унифициран PolicyReport. Така представени, резултатите от chaos експериментите се вписват в общия модел на Eirenyx за отчитане и могат да бъдат разглеждани по същия начин, както резултатите от Trivy и Falco.

За разлика от Trivy, който използва еднократни batch задачи, ресурсите ChaosEngine представляват персистентно декларативно състояние. Когато полето engineState е зададено на active, експериментите се активират незабавно и могат да бъдат изпълнявани периодично или при настъпване на определени условия. Това прави Litmus по-близък до модел на продължително управлявано поведение, отколкото до модел на еднократно изпълнявана задача.

Litmus политиката дефинира списък от експерименти, които трябва да бъдат изпълнени:

```
apiVersion: eirenyx.io/v1alpha1
kind: Policy
metadata:
  name: pod-resilience
  namespace: eirenyx-system
spec:
  type: litmus
  enabled: true
  litmus:
    experiments:
      - name: pod-delete-test
        experimentRef: pod-delete
    appInfo:
      appNamespace: production
```

```

    appLabel: "app=my-api"
    appKind: deployment
    duration: "60"
    mode: "Solo"
    targetNamespace: production
- name: cpu-stress-test
  experimentRef: pod-cpu-hog
  appInfo:
    appNamespace: production
    appLabel: "app=my-api"
    appKind: deployment
  duration: "120"
  mode: "Parallel"
  parameters:
    CPU_CORES: "2"
    CPU_LOAD: "80"

```

Основни параметри:

Поле	Описание
experimentRef	Тип на експеримента (ChaosExperiment)
appInfo	Целеви workload
duration	Продължителност
mode	Начин на изпълнение (Solo/Parallel)
parameters	Допълнителни параметри

Валидацията гарантира наличието на всички задължителни полета.

За всяка дефиниция на експеримент се създава отделен ChaosEngine ресурс. Името се генерира детерминистично:

```

func getChaosEngineName(policy *eirenyx.Policy, expName string) string {
    return fmt.Sprintf("eirenyx-litmus-%s-%s", policy.Name, expName)
}

```

Използва се `CreateOrUpdate`, което осигурява идиempотентност:

```
controllerutil.CreateOrUpdate(ctx, e.Client, engine, func() error {
    engine.Spec = litmus.ChaosEngineSpec{
        EngineState: "active",
        AppInfo: litmus.ApplicationParams{
            Appns:      exp.AppInfo.AppNamespace,
            Applabel: exp.AppInfo.AppLabel,
            AppKind:     exp.AppInfo.AppKind,
        },
        Experiments: []litmus.ExperimentList{{
            Name: exp.ExperimentRef,
            Spec: litmus.ExperimentAttributes{
                Components: litmus.ExperimentComponents{
                    ENV: buildEnvVars(exp),
                },
            },
        }},
    }
    return controllerutil.SetControllerReference(policy, engine, e.Scheme)
})
```

Функцията за генериране на `environment` променливи:

```
func buildEnvVars(exp eirenyx.LitmusExperiment) []corev1.EnvVar {
    envs := []corev1.EnvVar{}
    if exp.Duration != "" {
        envs = append(envs, corev1.EnvVar{Name: "TOTAL_CHAOS_DURATION",
            Value: exp.Duration})
    }
    if exp.Mode != "" {
        envs = append(envs, corev1.EnvVar{Name: "CHAOS_MODE", Value:
            exp.Mode})
    }
    for k, v := range exp.Parameters {
        envs = append(envs, corev1.EnvVar{Name: k, Value: v})
    }
    return envs
}
```


Важно уточнение: при cross-namespace изпълнение се изисква ръчно управление на cleanup, тъй като Kubernetes не поддържа cascading deletion между namespace-и.

LitmusReportHandler е по-опростен спрямо Trivy и Falco. Той изчаква резултати от ChaosResult и генерира отчет веднага след изпълнение на експериментите. Добавена е защита срещу излишни обновявания:

```
if !statusEqual(policyReport.Status, *newStatus) {  
    policyReport.Status = *newStatus  
    return h.Client.Status().Update(ctx, policyReport)  
}
```

Litmus допълва модела на сигурност:

Ниво	Инструмент	Функция
Pre-deployment	Trivy	Уязвимости
Runtime	Falco	Поведение
Resilience	Litmus	Устойчивост

Комбинирано това осигурява безопасни зависимости, нормално поведение в работната среда и устойчивост при отказ. Всички резултати от проведеният експеримент се представят чрез унифицирани Policy и PolicyReport ресурси. Фигура 7 илюстрира пълния процес на изпълнение и обработка на Litmus политики.

6. REST API и Уеб табло

Компонентът `eirenyx-api` представлява `stateless` HTTP сървър, реализиран на Go, който изпълнява ролята на междинен слой между уеб таблото на `Eirenyx` и `Kubernetes API`. Неговото основно предназначение е да предостави трите основни `Eirenyx CRD` ресурса – `Tool`, `Policy` и `PolicyReport` – като JSON-базиран REST интерфейс, достъпен за всеки HTTP клиент, без да се изисква директен достъп до `Kubernetes API` сървъра или познаване на вътрешния ресурсен модел на `Kubernetes`. Използването на отделен API компонент, вместо директно свързване на потребителския интерфейс към `Kubernetes API`, е обосновано от няколко конкретни архитектурни причини.

`Kubernetes API` сървърът използва сертификатна или `token`-базирана автентикация. Директното му излагане към `browser`-базиран потребителски интерфейс би изисквало или вграждане на `cluster credentials` в брауъра, което представлява сериозен риск за сигурността, или реализиране на сложни `OAuth/OIDC` механизми. Вместо това `eirenyx-api` се изпълнява вътре в клъстера като `Kubernetes pod` със собствен `ServiceAccount`, който притежава необходимите `RBAC` права. Брауърът комуникира с `eirenyx-api` чрез стандартен HTTP интерфейс, а API компонентът извършва заявките към `Kubernetes API` от името на потребителя. `Kubernetes` ресурсите съдържат значителен обем от метаданни, които са излишни или неразбираеми за `dashboard` потребител – например `managedFields`, `resourceVersion`, `creationTimestamp` и вътрешни `status conditions`. API слойът преобразува `Kubernetes` обектите в изчистени DTO структури, които съдържат само полетата, необходими за потребителския интерфейс. `Kubernetes API` връща грешки в собствен формат, със специфични `Kubernetes status` кодове и `reason` стойности. `eirenyx-api` преобразува тези грешки към стандартни HTTP статус кодове и унифицирана JSON структура, която може да бъде обработвана по един и същ начин от слоя за обработка на грешки в UI приложението. Ако `CRD` схемата на `Eirenyx` бъде променена, REST API слойът може да поеме тази промяна чрез DTO mapping логиката, без да се налага директна промяна в потребителския интерфейс. DTO слойът изпълнява ролята на граница между вътрешния модел на данните и външния API договор.

Избор на Go и chi

Компонентът `eirenyx-api` е реализиран на Go поради същите причини, поради които и операторът е реализиран на този език. Стандартната HTTP библиотека на Go е стабилна и подходяща за `production` среди, `client.Client` от `controller-runtime`, използван в оператора, може да бъде използван повторно и в API слоя без допълнителни зависимости, а моделът за компилация до един самостоятелен бинарен файл е особено подходящ за контейнеризирани `Kubernetes` приложения. [12] За HTTP маршрутизация е използван `chi` (github.com/go-chi/chi/v5). Той е избран пред алтернативни решения поради няколко причини:

- изграден е върху стандартната Go библиотека `net/http`, което означава, че всеки `middleware` или `handler`, съвместим с `net/http`, може да бъде използван без адаптация;
- предоставя ясно извличане на URL параметри чрез `chi.URLParam(r, "namespace")`, без необходимост от `reflection` или генериране на код;
- използва композиционен `middleware` модел, който позволява различни `middleware` компоненти да се прилагат към различни групи маршрути;

- основният пакет не въвежда нестандартни зависимости извън стандартната библиотека.

Пакетът `chi/middleware` предоставя готови за production употреба `middleware` компоненти като `RequestID`, `RealIP`, `Logger` и `Recoverer`, които иначе биха изисквали ръчна имплементация. Важна архитектурна особеност на `eirenyx-api` е, че той е скелетиран като `Kubebuilder controller manager`, а не като обикновен HTTP сървър. Входната точка `cmd/main.go` стартира `controller-runtime Manager`, регистрира `health probes` и стартира `chi HTTP сървър` като отделна `goroutine`, работеща паралелно с `manager` процеса. [12] Изгодният код може да се разгледа в Приложение 2.

Предимството на този подход е че тъй като HTTP сървърът получава `client.Client` от `controller manager`-а – същия кеширащ клиент, базиран на `informers`, който се използва и от `reconciler` компонентите в оператора. Този клиент не представлява обикновен Kubernetes REST клиент. Той разполага с локален `in-memory cache`, попълван чрез `list-and-watch informer` механизми, което означава, че множество `Get` и `List` операции могат да бъдат обслужени локално, без директно натоварване на Kubernetes API сървър. Така `eirenyx-api` автоматично се възползва от кеширащата инфраструктура на `controller-runtime`, без да изисква допълнителна конфигурация. При стартиране се регистрира `scheme`, който включва както стандартните Kubernetes типове (`clientgoscheme`), така и Eirenyx CRD типовете (`eirenyxv1alpha1`). Без регистрация и на двата набора от типове `client.Client` не би могъл да десериализира съответно нативните Kubernetes ресурси или персонализираните Eirenyx ресурси. `Graceful shutdown` е реализиран чрез `context.WithTimeout`, който предоставя на HTTP сървър до 10 секунди за завършване на текущите заявки при получаване на `SIGTERM`. Това гарантира, че при Kubernetes rolling update активните HTTP сесии няма да бъдат прекъснати неконтролирано.

Deployment в Kubernetes клъстер

Компонентът `eirenyx-api` се деплойма като Kubernetes Deployment с една реплика. Deployment манифестът, управляван чрез Kustomize, дефинира пълната оперативна конфигурация на компонента:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: controller-manager
  namespace: eirenyx-system
spec:
  replicas: 1
  template:
    spec:
      serviceAccountName: controller-manager
      securityContext:
        runAsNonRoot: true
        seccompProfile:
```

```

    type: RuntimeDefault
containers:
  - name: manager
    command: [ "/manager" ]
    args:
      - --leader-elect
      - --health-probe-bind-address=:8081
      - --http-bind-address=:8888
    image: controller:latest
    ports:
      - containerPort: 8888
        name: http
    securityContext:
      readOnlyRootFilesystem: true
      allowPrivilegeEscalation: false
      capabilities:
        drop: [ "ALL" ]
    resources:
      limits: { cpu: 500m, memory: 128Mi }
      requests: { cpu: 10m, memory: 64Mi }
    livenessProbe:
      httpGet: { path: /healthz, port: 8888 }
      initialDelaySeconds: 5
      periodSeconds: 10
    readinessProbe:
      httpGet: { path: /readyz, port: 8888 }
      initialDelaySeconds: 5
      periodSeconds: 10

```

В манифеста са приложени няколко важни мерки за повишаване на сигурността:

- `runAsNonRoot: true` – контейнерът се изпълнява с UID 65532 (nonroot потребителят от distroless базовия образ), а не като root. Това ограничава потенциалния обхват на компрометиране при container escape.
- `readOnlyRootFilesystem: true` – контейнерът няма право да записва върху собствената си файлова система. Тъй като бинарният файл няма нужда от запис на диск, тази мярка не въвежда функционална цена.
- `allowPrivilegeEscalation: false` – процесът не може да получи повече привилегии от тези, с които е стартиран, дори при наличие на уязвимост за изпълнение на произволен код.
- `capabilities.drop: ["ALL"]` – всички Linux capabilities са премахнати, тъй като API сървърът не се нуждае от специални kernel привилегии.

Контейнерният образ се изгражда чрез двустепенен Dockerfile: [15]

```
FROM golang:1.25 AS builder
WORKDIR /workspace
COPY go.mod go.sum .
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -a -o manager cmd/main.go

FROM gcr.io/distroless/static:nonroot
COPY --from=builder /workspace/manager .
USER 65532:65532
ENTRYPOINT ["/manager"]
```

Използването на `gcr.io/distroless/static:nonroot` като runtime базов образ има съществено значение. Distroless образите не съдържат shell, package manager, /tmp директория или стандартни системни инструменти. Атакуваната повърхност се свежда почти изцяло до Go бинарния файл, ако атакуващ получи изпълнение на код вътре в контейнера, той няма shell, помощни инструменти или мрежови utilities, чрез които лесно да разшири атаката или да извлече данни.

API сървърът се експонира в рамките на клъстера чрез Kubernetes Service от тип ClusterIP:

```
apiVersion: v1
kind: Service
metadata:
  name: http-service
  namespace: eirenyx-system
spec:
  type: ClusterIP
  selector:
    app.kubernetes.io/name: eirenyx-api
  ports:
    - name: http
      port: 80
      targetPort: 8888
```

Външният достъп може да бъде реализиран чрез Kubernetes Ingress или чрез service от тип LoadBalancer, в зависимост от конкретната клъстерна среда. В development среда локален достъп може да се осигури чрез port-forwarding, например:

```
kubect1 port-forward svc/http-service 8888:80
```

RBAC права

ServiceAccount-ът, асоцииран с eirenyx-api pod-a, използва ClusterRole със специфични права върху ресурсите, които API сървърът трябва да чете или модифицира:

```
rules:
- apiGroups: [ "eirenyx.eirenyx" ]
  resources: [ "tools", "policies" ]
  verbs: [ "create", "delete", "get", "list", "patch", "update", "watch" ]
- apiGroups: [ "eirenyx.eirenyx" ]
  resources: [ "policyreports" ]
  verbs: [ "get", "list", "watch" ]
- apiGroups: [ "eirenyx.eirenyx" ]
  resources: [ "tools/status", "policies/status", "policyreports/status" ]
  verbs: [ "get", "patch", "update" ]
- apiGroups: [ "batch" ]
  resources: [ "jobs" ]
  verbs: [ "create", "delete", "get", "list", "patch", "update", "watch" ]
```

Ресурсът PolicyReport умишлено е ограничен само до операции за четене (get, list, watch). Това ограничение се налага и на приложно ниво: няма дефинирани POST, PUT или DELETE endpoints за отчети. PolicyReport ресурсите се създават и управляват изключително от оператора и се прилага defense-in-depth подход, като ограничението съществува едновременно в API маршрутизацията и в RBAC правата.

Router и middleware

Функцията NewRouter приема client.Client от controller-runtime, инстанцира service и controller слоевете и конфигурира пълното дърво на маршрутите:

```
func NewRouter(k8s client.Client) http.Handler {
    r := chi.NewRouter()

    r.Use(middleware.RequestID)    // Unique ID on every request
    r.Use(middleware.RealIP)       // Extract real client IP from X-
```

```
Forwarded-For

r.Use(middleware.Logger)           // Structured access logging
r.Use(middleware.Recoverer)       // Recover from panics → 500

r.Use(cors.Handler(cors.Options{
    AllowedOrigins: []string{"*"},
    AllowedMethods: []string{"GET", "POST", "PUT", "PATCH", "DELETE",
"OPTIONS"},
    AllowedHeaders: []string{"Accept", "Authorization", "Content-
Type"},
    MaxAge:           300,
}))
}
```

CORS политиката (AllowedOrigins: ["*"]) е permissive, тъй като автентикацията е делегирана изцяло на Kubernetes RBAC. API сървърът не автентикира HTTP клиентите самостоятелно, а приема, че мрежовият достъп до pod-а е ограничен чрез Kubernetes NetworkPolicy. При среди, които изискват по-строг CORS контрол, допустимите origins могат да бъдат ограничени до конкретния hostname на UI приложението чрез Kustomize patch механизъм. Таблицата на маршрутите следва директно CRD йерархията:

Метод	Път	Действие
GET	/api/tools	Извличане на всички инструменти, с опционален ?namespace=
POST	/api/tools	Създаване на инструмент
GET	/api/tools/{namespace}/{name}	Извличане на конкретен инструмент
PUT	/api/tools/{namespace}/{name}	Замяна на спецификацията на инструмент
DELETE	/api/tools/{namespace}/{name}	Изтриване на инструмент
GET	/api/tools/{namespace}/{toolName}/policies	Извличане на политики, свързани с инструмент
GET	/api/policies	Извличане на всички политики
POST	/api/policies	Създаване на политика
GET	/api/policies/{namespace}/{name}	Извличане на конкретна

		политика
PUT	/api/policies/{namespace}/{name}	Замяна на спецификацията на политика
DELETE	/api/policies/{namespace}/{name}	Изтриване на политика
GET	/api/reports/{namespace}/{policy}	Извличане на отчети за дадена политика
GET	/api/reports/{namespace}/{policy}/{name}	Извличане на конкретен отчет

Обработка на грешки

Обработката на грешки е централизирана във файла `response.go`. Service слойът преобразува Kubernetes грешките в типизирани `sentinel errors` от пакета `api_errors`:

```
func (s *ToolService) Get(ctx context.Context, namespace, name string)
(*dto.Tool, error) {
    tool := &eirenyxv1alpha1.Tool{}
    if err := s.k8s.Get(ctx, types.NamespacedName{Namespace: namespace,
Name: name}, tool); err != nil {
        if k8serrors.IsNotFound(err) {
            return nil, api_errors.NewErrNotFound(fmt.Sprintf("tool %s/%s
not found", namespace, name))
        }
        return nil, fmt.Errorf("getting tool %s/%s: %w", namespace, name,
err)
    }
    return dto.MapToTool(*tool), nil
}
```

Функцията `respondError` в `controller` слоя преобразува тези типизирани грешки към HTTP статус кодове чрез `errors.As`:

```
func respondError(w http.ResponseWriter, err error) {
    switch {
    case errors.As(err, &notFound):
        statusCode = http.StatusNotFound // 404
    case errors.As(err, &conflict):
        statusCode = http.StatusConflict // 409
    case errors.As(err, &validation):
        statusCode = http.StatusUnprocessableEntity // 422
    }
```

```

default:
    statusCode = http.StatusInternalServerError // 500
}
respondJSON(w, statusCode, ErrorResponse{
    Error:    errorType,
    Message:  err.Error(),
    Code:     statusCode,
})
}

```

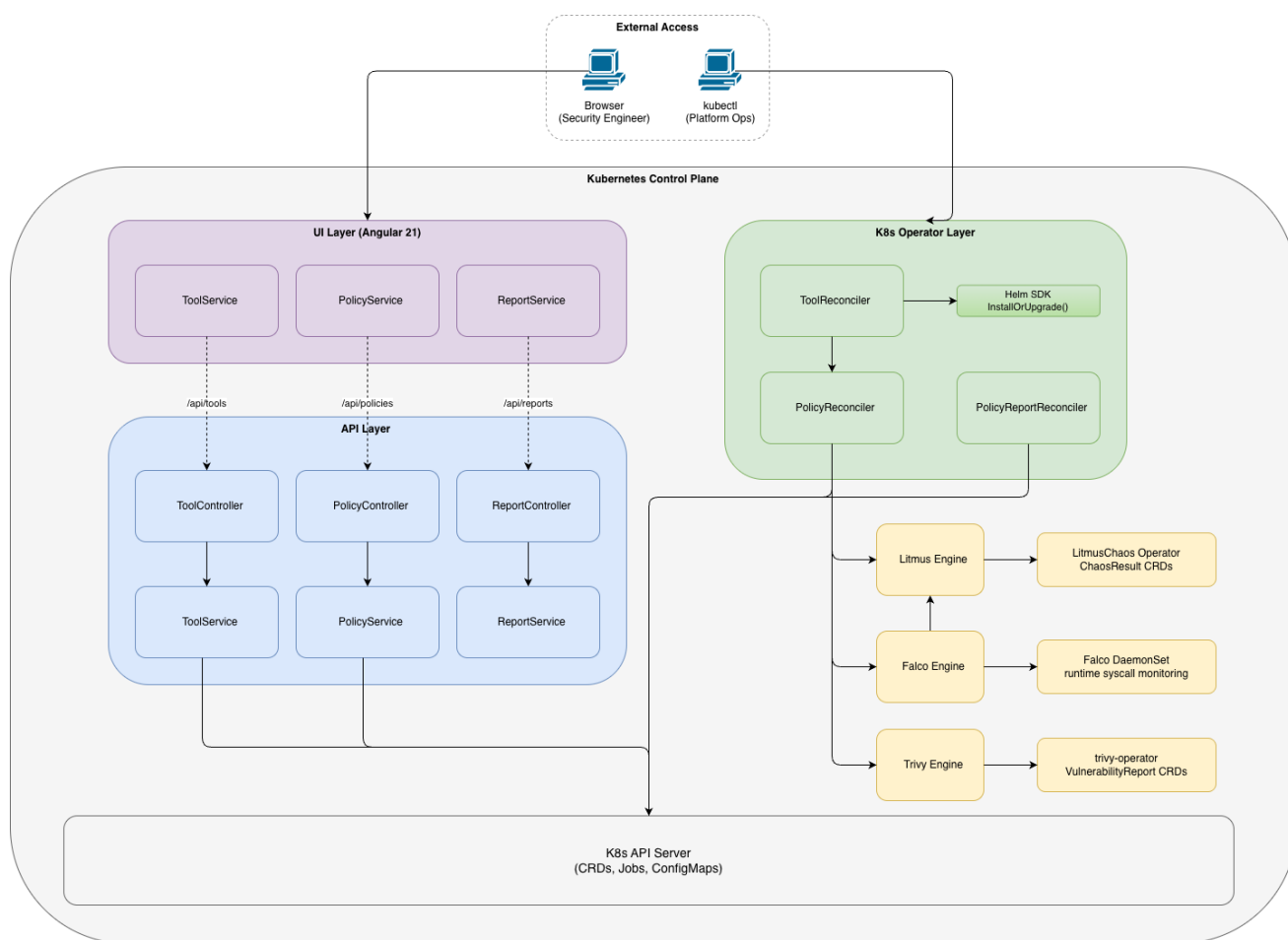
Използването на `errors.As`, вместо директен `type switch`, гарантира, че `wrapped errors` се разпознават коректно, което води до запазване на пълната `error chain` информация, като същевременно се генерира правилният HTTP отговор. Всички грешки използват единен JSON формат:

```

{
  "error": "Not Found",
  "message": "tool default/trivy not found",
  "code": 404
}

```

HTTP interceptor-ът в Angular UI прочита тази структура и използва полето `error`, за да извлече човешки разбираемо съобщение, което се визуализира в `notification toast` компонента. Консистентността на този формат позволява глобален `interceptor` модел — ако грешките имаха различни структури, всеки UI компонент би трябвало да реализира собствена логика за обработката им. Цялостното взаимодействие между потребителския интерфейс, REST API слоя, оператора и Kubernetes API е представено на Фигура 8. Диаграмата илюстрира как `eigenux-api` изпълнява ролята на посредник между уеб таблото и Kubernetes ресурсите, като запазва Kubernetes API като единствен източник на истина.



Фигура 8: Интеграция между компонентите и карта на интерфейсите

Компонентът `eirepux-ui` представлява single-page application (SPA), реализирано с Angular 21. То предоставя browser-базиран интерфейс за управление на ресурсите Tool и Policy, както и за преглед на резултатите, записани в PolicyReport. Приложението следва съвременната Angular архитектура със standalone компоненти и използва Signals API за реактивно управление на състоянието, без необходимост от външни библиотеки за state management. Приложението се стартира без използване на NgModule, като навсякъде се прилага standalone API моделът на Angular. Основната конфигурация регистрира два ключови provider-a:

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    provideHttpClient(withInterceptors([apiErrorInterceptor])),
  ]
};
```

Функцията `provideRouter(routes)` регистрира маршрутизацията на приложението, а `provideHttpClient(withInterceptors(...))` конфигурира HTTP клиента и глобалния HTTP interceptor за обработка на грешки. Това е функционалният interceptor API модел на Angular, който заменя по-стария подход с class-базирания интерфейс `HttpInterceptor` и не изисква отделна module декларация. Маршрутите са дефинирани чрез плоска конфигурация и използват `lazy-loaded standalone` компоненти. Всеки маршрут съответства директно на конкретен REST API endpoint. Параметрите от пътя, като `namespace`, `name` и `policy`, се извличат в компонентите чрез `inject(ActivatedRoute).snapshot.paramMap` и същите могат да работят с конкретен Kubernetes ресурс, без да познават вътрешната логика на API слоя.

HTTP комуникацията е реализирана чрез три injectable service класа – по един за всеки основен ресурсен тип. Всеки service е декориран с `@Injectable({ providedIn: 'root' })`, което го регистрира като singleton в root injector-а на Angular приложението и използването му във всички компоненти без необходимост от допълнителна module конфигурация. `ToolService` илюстрира общия модел:

```
@Injectable({providedIn: 'root'})
export class ToolService {
  private readonly base = environment.apiUrl;

  constructor(private readonly http: HttpClient) {
  }

  listTools(namespace?: string): Observable<Tool[]> {
    const params = namespace ? new HttpParams().set('namespace',
namespace) : undefined;
    return this.http.get<Tool[]>(`${this.base}/tools`, {params});
  }

  getTool(namespace: string, name: string): Observable<Tool> {
    return
this.http.get<Tool>(`${this.base}/tools/${namespace}/${name}`);
  }

  createTool(payload: ToolRequest): Observable<Tool> {
    return this.http.post<Tool>(`${this.base}/tools`, payload);
  }

  updateTool(namespace: string, name: string, payload: ToolRequest):
Observable<Tool> {
    return
this.http.put<Tool>(`${this.base}/tools/${namespace}/${name}`, payload);
  }
}
```

```

deleteTool(namespace: string, name: string): Observable<void> {
    return
this.http.delete<void>(`${this.base}/tools/${namespace}/${name}`);
}
}

```

Всички методи връщат `Observable<T>` с ясно дефиниран generic тип. Методите на Angular `HttpClient`, като `get<T>`, `post<T>`, `put<T>` и `delete<T>`, извършват автоматична JSON десериализация и типизират резултата. Това елиминира необходимостта от ръчно използване на `JSON.parse` и осигурява compile-time типова безопасност на границата между API слоя и frontend приложението. `PolicyService` разширява същия модел чрез поддръжка на опционални филтри:

```

listPolicies(namespace?: string, type?: string): Observable<Policy[]> {
    let params = new HttpParams();
    if (namespace) {
        params = params.set('namespace', namespace);
    }
    if (type) {
        params = params.set('type', type);
    }
    return this.http.get<Policy[]>(`${this.base}/policies`, {params});
}

```

`ReportService` е умишлено ограничен само до операции за четене. Той предоставя единствено методи като `listReports` и `getReport`, което съответства на read-only характера на `/reports` endpoints в REST API слоя. Това е приложено архитектурно решение, тъй като `PolicyReport` ресурсите се създават и управляват автоматично от оператора, а не от потребителския интерфейс.

HTTP Interceptor

В приложението се използва един глобален функционален interceptor – `apiErrorInterceptor`, който централизира обработката на всички HTTP грешки. Отделните компоненти не трябва да реализират дублирана логика за обработка на грешки.

```

export const apiErrorInterceptor: HttpInterceptorFn = (req, next) => {
    const notify = inject(NotificationService);

    return next(req).pipe(
        catchError((err: HttpErrorResponse) => {
            const message = err.error?.error ?? err.message;

```

```

        switch (err.status) {
            case 404:
                notify.error(`Not found: ${message}`);
                break;
            case 409:
                notify.error(`Conflict: ${message}`);
                break;
            case 422:
                notify.error(`Validation: ${message}`);
                break;
            default:
                notify.error(`Unexpected error: ${message}`);
        }

        return throwError(() => err);
    })
};

```

Interceptor-ът използва RxJS оператора `catchError`, за да прихване всяка HTTP грешка в заявките на приложението. [14] Той прочита полето `error` от JSON структурата, върната от API слоя (`err.error?.error`), и изпраща съобщение към `NotificationService`. След визуализиране на съобщението грешката се хвърля повторно чрез `throwError`. Този подход позволява на конкретни компоненти да реагират на специфични ситуации, например пренасочване към списъчна страница при 404 Not Found, без да се нарушава глобалната обработка на грешки. Използването на `inject()` във функционален `HttpInterceptorFn` е идиоматичен подход в Angular 21. Функцията се изпълнява в `dependency injection context`, което позволява `inject(NotificationService)` да достъпи `singleton` инстанцията на услугата без `constructor injection`.

Компонентите управляват локалното си състояние чрез Angular Signals API. Signals представляват синхронни реактивни стойности, които автоматично уведомяват зависимите изчисления и шаблони при промяна, от което следва реактивно обновяване на интерфейса без ръчно задействане на `change detection` и без необходимост от сложна RxJS state management логика. Компонентът `ToolDetail` демонстрира този модел:

```

@Component({
    selector: 'app-tool-detail',
    standalone: true,
    // ...
})
export class ToolDetail implements OnInit {
    tool = signal<Tool | null>(null);
}

```

```

loading = signal(true);
deleting = signal(false);

private readonly api = inject(ToolService);
private readonly notify = inject(NotificationService);
private readonly route = inject(ActivatedRoute);
private readonly router = inject(Router);

ngOnInit(): void {
    this.api.getTool(this.namespace, this.name).subscribe({
        next: tool => this.tool.set(tool),
        error: err => this.notify.apiError(err, `Failed to load
tool.`),
        complete: () => this.loading.set(false),
    });
}
}

```

Всяко отделно състояние е представено като независим signal. Например loading управлява визуализирането на spinner или skeleton елемент, deleting може да деактивира бутона за изтриване при активна заявка, а tool съдържа заредения ресурс. Шаблонът прочита тези стойности директно чрез извикване на signal функцията, а Angular автоматично обновява само зависимите части от интерфейса при промяна. Компонентът ToolsList показва как signals се обновяват на база резултати от Observable subscriptions:

```

tools = signal<Tool[]>([]);

private load(): void {
    this.api.listTools().subscribe({
        next: tools => this.tools.set(tools),
        error: err => this.notify.apiError(err, 'Failed to load tools.'),
        complete: () => this.loading.set(false),
    });
}

delete(tool: Tool): void {
    this.api.deleteTool(tool.namespace, tool.name).subscribe({
        next: () => {
            this.tools.update(all => all.filter(t => t.name !==
tool.name));
            this.notify.success(`Tool deleted successfully.`);
        }
    });
}

```

```
    },  
    });  
}
```

След успешно изтриване методът `signal.update()` прилага трансформация върху текущата стойност на списъка, като премахва изтрития ресурс. Потребителският интерфейс се обновява оптимистично, без повторно зареждане на целия списък от API.

IV. ЗАКЛЮЧЕНИЕ, ОГРАНИЧЕНИЯ И БЪДЕЩА РАБОТА

1. Обобщение на постигнатите резултати

Настоящата дипломна работа е насочена към решаването на ясно дефиниран проблем в екосистемата от инструменти за сигурност в Kubernetes: липсата на open-source, Kubernetes-native оператор, който предоставя унифициран декларативен интерфейс за управление на пълния жизнен цикъл на инструменти за сигурност и устойчивост – от инсталация, през конфигурация на политики, до нормализирано отчитане на резултатите – без да заменя използваните инструменти и без да изисква проприетарна cloud инфраструктура. Шестте изисквания, предоставят рамката за оценка на постигнатите резултати.

R1 – Неинвазивна интеграция

Eirenyx управлява Trivy, Falco и LitmusChaos без да модифицира техния изходен код, Helm chart-ове или вътрешни CRD схеми. Всеки инструмент продължава да функционира самостоятелно, като операторът добавя управляващ слой върху тях. Trivy продължава да генерира VulnerabilityReport CRD ресурси чрез собствения си оператор; Falco продължава да изпълнява своя механизъм за правила на ниво ядро; Litmus продължава да изпълнява експерименти чрез собствения си chaos-operator. Eirenyx действа като потребител и оркестратор на тези инструменти, а не като техен заместител. И трите инструмента се инсталират и управляват чрез Helm Go SDK, който работи с официалните upstream chart-ове. В контейнерния образ на оператора не е включен инструмент-специфичен бинарен файл.

R2 – Декларативно управление на жизнения цикъл

Tool CRD и ToolReconciler осигуряват изцяло декларативно управление на жизнения цикъл и за трите инструмента. Създаването на ресурс от тип Tool води до инсталиране на съответния Helm chart; задаването на spec.enabled: false води до неговото деинсталиране; изтриването на Tool обекта задейства контролирано почистване на Helm release-а чрез finalizer pattern. В оператора не се използват kubectl команди, Helm CLI извиквания или shell скриптове. Принципът на разработка „без извикване на външни CLI инструменти“ е спазен в цялата имплементация.

R3 – Унифициран модел за политики

Policy CRD предоставя единна входна точка за конфигуриране на всички три инструмента. Trivy сканиране за уязвимости, Falco правило за runtime детекция и Litmus chaos експеримент се описват като ресурси от тип Policy с обща структура на най-горно ниво (type, enabled, target) и инструмент-специфични подспецификации (spec.trivy, spec.falco, spec.litmus). Factory pattern и трите реализации на Engine интерфейса (TrivyEngine, FalcoEngine, LitmusEngine) осигуряват инструмент-специфичния слой за изпълнение, като същевременно PolicyReconciler остава напълно независим от конкретния инструмент. Платформеният инженер конфигурира и трите инструмента чрез един и същ интерфейс.

R4 – Нормализирана отчетност

И трите инструмента генерират PolicyReport ресурси с обща схема: фаза на изпълнение, обобщение (Summary) с бинарен резултат Verdict (Pass/Fail), броячи TotalChecks, Passed и Failed, както и поле Details, съдържащо инструмент-специфични резултати. Trivy и Falco report handler-ите генерират отчети с реален verdict и пълни данни за резултатите. Litmus report handler-ът генерира отчет, който потвърждава планирането на експериментите. Корелацията между резултатите от различните инструменти е идентифицирана като бъдеща работа.

R5 – Kubernetes-native състояние

Нито един компонент на Eirenyx не използва външна база данни, конфигурационни файлове на диск или вътрешно in-memory състояние като източник на истина. Цялото персистентно състояние – намерение за инсталация на инструмент, конфигурация на политики и резултати от проверки – се съхранява в Custom Resources в etcd. Операторът, API компонентът и UI приложението могат да бъдат рестартирани по всяко време без загуба на данни. Кеширащият клиент на controller-runtime гарантира, че повтарящите се read операции не водят до излишно натоварване върху Kubernetes API сървър. Това изискване е изпълнено чрез трислойния CRD модел и използването на controller-runtime клиента във всички компоненти. [5]

R6 – Съвместимост с GitOps

Тъй като цялата конфигурация на Eirenyx се описва чрез Kubernetes YAML манифести, цялостното състояние на сигурността на клъстера – кои инструменти са инсталирани, кои политики са активни, кои образи се сканират, кои Falco правила се наблюдават и кои chaos експерименти са планирани – може да бъде версионизирано в Git хранилище и прилагано чрез GitOps контролери като Flux или ArgoCD. Промяната на политиката за сигурност може да се осъществява чрез pull request към конфигурационното хранилище, което осигурява пълна проследимост на промените за екипите по сигурност. И трите CRD типа са напълно декларативни и могат да бъдат сериализирани като YAML. Принципът „декларативно пред императивно“ е спазен в цялата имплементация.

2. Архитектурни приноси

Освен покритието на потребителските истории, реализираната система предоставя няколко архитектурни приноса, които имат стойност независимо от конкретните три интегрирани инструмента.

Интерфейсите ToolService, Engine и Handler дефинират минимален договор за интеграция на произволен инструмент за сигурност или устойчивост в рамката на Eirenyx. Тези три интерфейса заедно описват пълния жизнен цикъл на управляван инструмент: инсталация (ToolService), изпълнение на политика (Engine) и събиране на резултати (Handler). Нов инструмент – например скенер за network policies, верификатор на image signatures или SBOM генератор – може да бъде интегриран чрез имплементиране на тези три интерфейса и добавяне на съответните case разклонения във factory функциите. Не се налага промяна в controller слоя. Тази разширяемост чрез конвенция представлява основния архитектурен принос на системата.

Използването на `metadata.generation` като сигнал за промяна при инвалидиране на `PolicyReport` – чрез сравнение между текущата `generation` стойност на политиката и `generation` стойността, записана в спецификацията на отчета – предоставя чист и устойчив механизъм за гарантиране, че резултатите от сканиране винаги съответстват на текущата конфигурация на политиката. Този модел може да бъде използван повторно във всеки Kubernetes оператор, който трябва да инвалидира производни ресурси при промяна на техния източник.

Концентрирането на цялата логика за избор на инструмент в три `factory` функции (`NewToolService`, `NewPolicyEngine`, `NewReportEngine`) гарантира, че контролерите остават напълно свободни от условна логика, базирана на типа инструмент. Това е директно приложение на принципа `Open/Closed` на ниво Kubernetes оператор: системата е затворена за модификация в `controller` слоя и отворена за разширение чрез нови `factory` случаи и имплементации на интерфейси.

3. Ограничения

Корелиране на резултатите от Trivy, Falco и Litmus с цел създаване на унифицирана оценка на риска за конкретен workload не е реализирано. В момента всеки `PolicyReport` обхваща резултатите само на един инструмент. Workload, който има Trivy CVE под прага за аларма, активен Falco alert и неуспешен Litmus експеримент, не може автоматично да бъде идентифициран като високорисков в рамките на Eirenyx. Трите сигнала съществуват като отделни обекти, без програмна връзка помежду им, освен че могат да се намират в един и същ namespace. Сценарият за интеграционна празнина при който комбинацията от сигнали е по-значима от всеки отделен сигнал не може да бъде автоматично засечен от текущата имплементация.

`PolicyReconciler` намира Tool ресурса, към който принадлежи дадена Policy, като приема, че името на инструмента съвпада с неговия тип в рамките на същия namespace. Например политика със `spec.type: trivy` очаква Tool с име `trivy` в същия namespace. Тази naming convention е проста и елиминира необходимостта от foreign-key reference поле в `PolicySpec`, но води до следните ограничения:

- само една инстанция от всеки тип инструмент може да бъде активна в рамките на един namespace;
- политиките не могат да реферират инструменти в други namespace-и;
- преименуването на Tool ресурс нарушава всички политики от този тип.

Това ограничение е приемливо при `single-cluster`, `single-team` среди, но се превръща в ограничение при `multi-tenant` среди, където различни екипи може да се нуждаят от различни Trivy конфигурации в един и същ namespace.

`eirenyx-api` е конфигуриран с `AllowedOrigins: ["*"]`, което означава, че приема заявки от произволен origin. Това е приемливо, когато мрежовият достъп до API pod-а е ограничен чрез Kubernetes NetworkPolicy, но представлява повърхност за атака, която трябва да бъде ограничена в production среди. Текущата имплементация не предоставя конфигурационен механизъм за ограничаване на допустимите origins без директна промяна на Kustomize манифестите. Уеб таблото също делегира автентикацията изцяло на Kubernetes RBAC системата, като разчита на ServiceAccount идентичността на pod-а, в който се изпълнява. Макар този подход да е архитектурно чист, той означава, че към момента няма механизъм за multi-user access control на API ниво. Всеки процес, който

има мрежов достъп до API pod-a, получава пълните права на неговия ServiceAccount. Production deployment би имал полза от OIDC-базирана автентикация, при която bearer token-ът на потребителя се препраща към TokenReview endpoint-a на Kubernetes API сървър.

4. Бъдеща работа

Описаните ограничения формират естествена пътна карта за бъдещо развитие. Изисква се модел за корелация, който свързва резултати от различни инструменти към обща идентичност на workload. Практическа имплементация би могла да включва:

- въвеждане на WorkloadReport CRD, който агрегира PolicyReport обекти, насочени към един и същ workload, идентифициран чрез namespace и label selector;
- имплементиране на WorkloadReportReconciler, който изчислява composite risk score на база verdict-ите на отделните инструменти, претеглени спрямо сериозността;
- експониране на WorkloadReport чрез REST API и визуализацията му в отделна dashboard страница, която показва трислойната security posture оценка на всеки workload.

Това би адресирало multi-signal risk сценария: workload със средна CVE уязвимост, Falco alert и неуспешен Litmus тест би получил composite risk оценка High, въпреки че нито един от отделните сигнали самостоятелно не преминава прага за аларма.

Текущата имплементация изпълнява Trivy сканирания on-demand, при създаване на Policy или при промяна на нейната спецификация. Полезно разширение би било добавяне на поле schedule в TrivyPolicySpec, използващо стандартен cron синтаксис, което да задейства периодични повторни сканирания. Това е необходимо, тъй като нови CVE уязвимости се публикуват непрекъснато: образ, който е преминал успешно сканиране в понеделник, може да се окаже рисков в четвъртък, ако бъде публикувана нова уязвимост в някоя от неговите зависимости. Планираното сканиране би приближило Eirenyx до continuous scanning модела на trivy-operator, като същевременно запази policy-driven control plane подхода.

Замяната на имплицитната naming convention, при която политика със spec.type: trivy се свързва с Tool с име trivy, с изрично поле spec.toolRef.name в PolicySpec би премахнала ограничението за една инстанция на инструмент от даден тип в рамките на namespace. Това би позволило multi-tenant deployment-и, при които различни екипи поддържат отделни инстанции на инструментите с различни конфигурации в един и същ клъстер, а политиките могат експлицитно да реферират конкретната инстанция, която трябва да използват.

Разширяване на eirenyx-api с OIDC автентикация, чрез препращане на bearer token-a на потребителя към Kubernetes API TokenReview endpoint-a би осигурило per-user access control без въвеждане на отделен identity provider. Този модел, използван от Kubernetes Dashboard и инструменти като Headlamp, позволява съществуващият Kubernetes RBAC да контролира кои потребители могат да създават или изтриват Tool и Policy ресурси, вместо целият достъп да бъде предоставен чрез ServiceAccount-a на API pod-a.

Factory pattern и моделът с трите интерфейса са проектирани с цел разширяемост. Потенциални инструменти за бъдеща интеграция са изброени в таблицата по долу:

Инструмент	Предназначение	Тип интеграция
Kyverno	Admission control и policy enforcement	Engine създава Policy обекти; Handler чете PolicyReport
Cosign / Sigstore	Проверка на подписи на контейнерни образи	Engine стартира job за проверка на подпис; Handler чете резултата
OWASP ZAP	Dynamic application security testing	Engine създава scan job; Handler чете JSON резултати
Terrascan	Сканиране за неправилни конфигурации в Infrastructure-as-Code	Engine създава job срещу manifest repository; Handler чете SARIF output

Всеки от тези инструменти може да бъде интегриран чрез добавяне на нова ToolType константа, имплементиране на трите интерфейса и добавяне на три case разклонения във factory функциите – без промени в controller слоя.

Текущата логика за verdict е hardcoded: Trivy връща Fail при CRITICAL/HIGH, а Falco връща Fail при eventCount > 0. Добавянето на конфигурационно поле VerdictPolicy във всяка инструмент-специфична подспецификация би позволило на platform екипите да настройват тези прагове. Например в production среда може да се третира като Fail само CRITICAL, докато в development среда като Fail могат да се третират CRITICAL/HIGH/MEDIUM. Това би намалило alert fatigue в по-малко критични среди, като същевременно запази строг контрол там, където е необходим.

5. Заключение бележки

Eirenuх демонстрира, че е възможно да се изгради унифицирана open-source контролна равнина за инструменти за сигурност в Kubernetes, която удовлетворява основните изисквания – декларативно управление на жизнения цикъл, унифицирана конфигурация на политики, нормализирана отчетност и GitOps съвместимост, без да заменя или fork-ва използваните инструменти, без да изисква проприетарна cloud инфраструктура, и без да въвежда външно хранилище на състоянието. Трислойният CRD модел (Tool, Policy, PolicyReport), factory pattern и абстракцията чрез трите интерфейса формират framework, който е едновременно практически приложим и лесно разширяем. Шестте epics, дефинирани в началото на проекта, са реализирани в значителна степен:

- управлението на жизнения цикъл на инструментите

- reconciliation на политиките
- инфраструктурата за отчетност

Тези точки са имплементирани и функциониращи. По-широкият принос на разработката е представянето на референтна архитектура за модела „operator-as-integration-layer“ е използване на Kubernetes operator pattern не за управление на едно-единствено сложно приложение, а за предоставяне на унифициран декларативен интерфейс върху хетерогенна група от вече съществуващи инструменти. Този модел е приложим и извън областта на сигурността, във всяка ситуация, в която множество независимо разработени Kubernetes-native инструменти трябва да бъдат управлявани като единна система от екип, който не следва да бъде принуден да познава вътрешния модел на всеки отделен инструмент.

V. ИСПОЛЗВАНИ ЛИТЕРАТУРНИ ИЗТОЧНИЦИ

1. Kubernetes документация: <https://kubernetes.io/docs/concepts/>
2. Cloud Native Computing Fondation (CNCF): <https://www.cncf.io/>
3. Operator Framework: <https://operatorframework.io/operator-capabilities/>
4. Kubebuilder книга: <https://book.kubebuilder.io/>
5. controller-runtime репозитори: <https://github.com/kubernetes-sigs/controller-runtime>
6. controller-gen документация: <https://book.kubebuilder.io/reference/controller-gen.html>
7. Helm документация: <https://helm.sh/docs/>
8. Trivy документация: <https://trivy.dev/>
9. Aqua Security документация: <https://www.aquasec.com/>
10. Falco документация: <https://falco.org/docs/>
11. Litmus Chaos документация: <https://docs.litmuschaos.io/>
12. Chi рутер на GoLang: <https://github.com/go-chi/chi>
13. Angular Signals API: <https://angular.dev/guide/signals>
14. RxJS документация: <https://rxjs.dev/>
15. Docker документация: <https://docs.docker.com/>

VI. ПРИЛОЖЕНИЕ

Приложение 1a – Исходен код за реконсiliaция на инструмент (Tool)

```
package controller

import (
    "context"
    "time"

    "github.com/EirenyxK8s/eirenyx/internal/client/k8s"
    "k8s.io/apimachinery/pkg/api/meta"
    metav1 "k8s.io/apimachinery/pkg/apis/meta/v1"
    "k8s.io/apimachinery/pkg/runtime"
    ctrl "sigs.k8s.io/controller-runtime"
    "sigs.k8s.io/controller-runtime/pkg/client"
    "sigs.k8s.io/controller-runtime/pkg/controller/controllerutil"
    logf "sigs.k8s.io/controller-runtime/pkg/log"

    eirenyx "github.com/EirenyxK8s/eirenyx/api/v1alpha1"
)

// ToolReconciler reconciles a Tool object
type ToolReconciler struct {
    client.Client
    Scheme      *runtime.Scheme
    K8sClient k8s.Client
}

func (r *ToolReconciler) Reconcile(ctx context.Context, req ctrl.Request) (ctrl.Result, error) {
    log := logf.FromContext(ctx)
    log.Info("Starting Tool Reconciliation")

    var tool eirenyx.Tool
    if err := r.Get(ctx, req.NamespacedName, &tool); err != nil {
        return CompleteWithError(client.IgnoreNotFound(err))
    }

    if !tool.DeletionTimestamp.IsZero() {
```



```

        return r.handleDeletion(ctx, &tool)
    }

    if !controllerutil.ContainsFinalizer(&tool, eirenyx.ToolFinalizer) {
        patch := client.MergeFrom(tool.DeepCopy())
        controllerutil.AddFinalizer(&tool, eirenyx.ToolFinalizer)
        if err := r.Patch(ctx, &tool, patch); err != nil {
            return CompleteWithError(err)
        }
    }

    svc, err := NewToolService(&tool, Dependencies{
        Client:    r.Client,
        Scheme:    r.Scheme,
        K8sClient: &r.K8sClient,
    })
    if err != nil {
        return r.setFailedCondition(ctx, &tool, "UnsupportedToolType",
err.Error())
    }

    log.Info("Reconciling Tool", "service", svc.Name(), "enabled",
tool.Spec.Enabled)

    if tool.Spec.Enabled {
        if err := svc.EnsureInstalled(ctx, &tool); err != nil {
            log.Error(err, "Failed to install tool")
            return RequeueAfter(5 * time.Second)
        }
    } else {
        if err := svc.EnsureUninstalled(ctx, &tool); err != nil {
            log.Error(err, "Failed to uninstall tool")
            return RequeueAfter(5 * time.Second)
        }
    }

    healthy, err := svc.CheckHealth(ctx, &tool)
    if err != nil {
        log.Error(err, "Health check failed")
        return RequeueAfter(10 * time.Second)
    }

```

```

    tool.Status.Installed = tool.Spec.Enabled
    tool.Status.Healthy = healthy
    if err := r.Status().Update(ctx, &tool); err != nil {
        log.Error(err, "Failed to update tool status")
        return RequeueAfter(5 * time.Second)
    }

    if !healthy {
        log.Info("Tool not yet healthy, requeuing", "tool", tool.Name)
        return RequeueAfter(5 * time.Second)
    }

    log.Info("Finished Tool Reconciliation", "tool", tool.Name)
    return Complete()
}

func (r *ToolReconciler) handleDeletion(ctx context.Context, tool
*eirenyx.Tool) (ctrl.Result, error) {
    log := logf.FromContext(ctx)
    log.Info("Tool is being deleted", "tool", tool.Name)

    svc, err := NewToolService(tool, Dependencies{
        Client:    r.Client,
        Scheme:    r.Scheme,
        K8sClient: &r.K8sClient,
    })
    if err != nil {
        log.Info("Unknown tool type during deletion, skipping
uninstall", "type", tool.Spec.Type)
    } else {
        if err := svc.EnsureUninstalled(ctx, tool); err != nil {
            log.Error(err, "Failed to uninstall tool during
deletion")
            return RequeueAfter(10 * time.Second)
        }
    }

    patch := client.MergeFrom(tool.DeepCopy())
    controllerutil.RemoveFinalizer(tool, eirenyx.ToolFinalizer)
    if err := r.Patch(ctx, tool, patch); err != nil {

```

```

        return CompleteWithError(err)
    }

    log.Info("Finalizer removed, deletion completed", "tool", tool.Name)
    return Complete()
}

func (r *ToolReconciler) setFailedCondition(ctx context.Context, tool
*eirenyx.Tool, reason, message string) (ctrl.Result, error) {
    log := logf.FromContext(ctx)
    log.Error(nil, message, "tool", tool.Name)

    meta.SetStatusCondition(&tool.Status.Conditions, metav1.Condition{
        Type:            "Ready",
        Status:          metav1.ConditionFalse,
        Reason:          reason,
        Message:         message,
        LastTransitionTime: metav1.Now(),
    })

    if err := r.Status().Update(ctx, tool); err != nil {
        log.Error(err, "Failed to update status condition")
        return RequeueAfter(5 * time.Second)
    }

    return Complete()
}

func (r *ToolReconciler) SetupWithManager(mgr ctrl.Manager) error {
    return ctrl.NewControllerManagedBy(mgr).
        For(&eirenyx.Tool{}).
        Named("tool").
        Complete(r)
}

```

Приложение 1b – Исходен код за реконсиляция на политика (Policy)

```
package controller

import (
    "context"
    "fmt"
    "time"

    "github.com/pkg/errors"
    "k8s.io/apimachinery/pkg/runtime"
    ctrl "sigs.k8s.io/controller-runtime"
    "sigs.k8s.io/controller-runtime/pkg/client"
    "sigs.k8s.io/controller-runtime/pkg/controller/controllerutil"
    logf "sigs.k8s.io/controller-runtime/pkg/log"

    eirenyx "github.com/EirenyxK8s/eirenyx/api/v1alpha1"
)

// PolicyReconciler reconciles a Policy object
type PolicyReconciler struct {
    client.Client
    Scheme *runtime.Scheme
}

func (r *PolicyReconciler) Reconcile(ctx context.Context, req ctrl.Request) (ctrl.Result, error) {
    log := logf.FromContext(ctx)
    log.Info("Starting Policy Reconciliation")

    var policy eirenyx.Policy
    if err := r.Get(ctx, req.NamespacedName, &policy); err != nil {
        return CompleteWithError(client.IgnoreNotFound(err))
    }

    log.Info("Reconciling Policy With Spec: ", "policySpec", policy.Spec)

    var tool eirenyx.Tool
    if err := r.Get(ctx, client.ObjectKey{
        Name:      string(policy.Spec.Type),
        Namespace: policy.Namespace,
    }); err != nil {
        return CompleteWithError(client.IgnoreNotFound(err))
    }
}
```

```

    }, &tool); err != nil {
        message := fmt.Sprintf("failed to get tool %q from namespace
%q.", policy.Spec.Type, policy.Namespace)
        return CompleteWithError(errors.Wrap(err, message))
    }

    engine, err := NewPolicyEngine(&policy, Dependencies{
        Client: r.Client,
        Scheme: r.Scheme,
    })
    if err != nil {
        return CompleteWithError(err)
    }

    if !policy.DeletionTimestamp.IsZero() {
        log.Info("Policy is being deleted", "policy", policy.Name)

        if controllerutil.ContainsFinalizer(&policy,
eirenyx.PolicyFinalizer) {
            if err := engine.Cleanup(ctx, &policy); err != nil {
                log.Error(err, "Failed to cleanup policy")
                return RequeueAfter(time.Second * 5)
            }

            controllerutil.RemoveFinalizer(&policy,
eirenyx.PolicyFinalizer)
            if err := r.Update(ctx, &policy); err != nil {
                return CompleteWithError(err)
            }
        }

        log.Info("Finalizer removed, deletion completed", "policy",
policy.Name)
        return Complete()
    }

    if !controllerutil.ContainsFinalizer(&policy,
eirenyx.PolicyFinalizer) {
        controllerutil.AddFinalizer(&policy, eirenyx.PolicyFinalizer)
        if err := r.Update(ctx, &policy); err != nil {
            return CompleteWithError(err)
        }
    }

```

```

        return Complete()
    }

    has, err := controllerutil.HasOwnerReference(policy.OwnerReferences,
&tool, r.Scheme)
    if err != nil {
        return CompleteWithError(err)
    }

    if !has {
        if err := controllerutil.SetOwnerReference(&tool, &policy,
r.Scheme); err != nil {
            return CompleteWithError(err)
        }
        if err := r.Update(ctx, &policy); err != nil {
            return CompleteWithError(err)
        }
        return Complete()
    }

    if !policy.Spec.Enabled {
        if err := engine.Cleanup(ctx, &policy); err != nil {
            log.Error(err, "Failed to cleanup policy")
            return RequeueAfter(time.Second * 5)
        }
        return Complete()
    }

    if err := engine.Validate(&policy); err != nil {
        return CompleteWithError(err)
    }

    if err := engine.Reconcile(ctx, &policy); err != nil {
        log.Error(err, "Failed to reconcile policy")
        return RequeueAfter(time.Second * 5)
    }
    log.Info("Policy reconciled successfully")

    report, err := engine.GenerateReport(ctx, &policy)
    if err != nil {
        log.Error(err, "Failed to generate Policy Report")
    }

```

```

        return CompleteWithError(err)
    }

    needsRescan := report.Spec.PolicyRef.Generation != policy.Generation

    if _, err := controllerutil.CreateOrUpdate(ctx, r.Client, report,
func() error {
        report.Spec.PolicyRef.Name = policy.Name
        report.Spec.PolicyRef.Generation = policy.Generation
        report.Spec.Type = policy.Spec.Type
        return nil
    }); err != nil {
        return CompleteWithError(err)
    }

    if needsRescan {
        log.Info("Policy generation changed, resetting report to
Pending for re-scan",
            "policy", policy.Name,
            "oldGeneration", report.Spec.PolicyRef.Generation,
            "newGeneration", policy.Generation,
        )
        report.Status.Phase = eirenyx.ReportPending
        report.Status.Summary = eirenyx.ReportSummary{}
        report.Status.Details = runtime.RawExtension{}
        if err := r.Status().Update(ctx, report); err != nil {
            return CompleteWithError(err)
        }
    }

    policy.Status.LastReport = report.Name
    policy.Status.ObservedGen = policy.Generation
    if err := r.Status().Update(ctx, &policy); err != nil {
        return CompleteWithError(err)
    }

    log.Info("Finished Policy Reconciliation", "policy", policy.Name)
    return Complete()
}

// SetupWithManager sets up the controller with the Manager.

```

```

func (r *PolicyReconciler) SetupWithManager(mgr ctrl.Manager) error {
    return ctrl.NewControllerManagedBy(mgr).
        For(&eirenyx.Policy{}).
        Named("policy").
        Complete(r)
}

```

Приложение 2 – Входна точка на eirenyx-api

```

package main

import (
    "context"
    "crypto/tls"
    "errors"
    "flag"
    "net/http"
    "os"
    "time"

    "github.com/EirenyxK8s/eirenyx-api/internal/mvc"
    _ "k8s.io/client-go/plugin/pkg/client/auth"

    "k8s.io/apimachinery/pkg/runtime"
    utilruntime "k8s.io/apimachinery/pkg/util/runtime"
    clientgoscheme "k8s.io/client-go/kubernetes/scheme"

    eirenyxv1alpha1 "github.com/EirenyxK8s/eirenyx/api/v1alpha1"
    ctrl "sigs.k8s.io/controller-runtime"
    "sigs.k8s.io/controller-runtime/pkg/healthz"
    "sigs.k8s.io/controller-runtime/pkg/log/zap"
    "sigs.k8s.io/controller-runtime/pkg/metrics/filters"
    metricsserver "sigs.k8s.io/controller-runtime/pkg/metrics/server"
    "sigs.k8s.io/controller-runtime/pkg/webhook"
    // +kubebuilder:scaffold:imports
)

var (
    scheme = runtime.NewScheme()
    setupLog = ctrl.Log.WithName("setup")

```



```

)

func init() {
    utilruntime.Must(clientgoscheme.AddToScheme(scheme))
    utilruntime.Must(eirenyxv1alpha1.AddToScheme(scheme))

    // +kubebuilder:scaffold:scheme
}

// nolint:gocyclo
func main() {
    var metricsAddr string
    var metricsCertPath, metricsCertName, metricsCertKey string
    var webhookCertPath, webhookCertName, webhookCertKey string
    var enableLeaderElection bool
    var probeAddr string
    var secureMetrics bool
    var enableHTTP2 bool
    var httpAddr string

    var tlsOpts []func(*tls.Config)

    flag.StringVar(&metricsAddr, "metrics-bind-address", "0", "The
address the metrics endpoint binds to. "+
        "Use :8443 for HTTPS or :8080 for HTTP, or leave as 0 to
disable the metrics service.")

    flag.StringVar(&probeAddr, "health-probe-bind-address", ":8081", "The
address the probe endpoint binds to.")

    flag.StringVar(&httpAddr, "http-bind-address", ":8888", "The address
the HTTP API server binds to.")

    flag.BoolVar(&enableLeaderElection, "leader-elect", false,
        "Enable leader election for controller manager. "+
            "Enabling this will ensure there is only one active
manager instance.")

    flag.BoolVar(&secureMetrics, "metrics-secure", true,
        "If set, the metrics endpoint is served securely via HTTPS. Use
--metrics-secure=false to use HTTP instead.")

    flag.StringVar(&webhookCertPath, "webhook-cert-path", "", "The
directory that contains the webhook certificate.")

    flag.StringVar(&webhookCertName, "webhook-cert-name", "tls.crt", "The
name of the webhook certificate file.")

    flag.StringVar(&webhookCertKey, "webhook-cert-key", "tls.key", "The
name of the webhook key file.")

```

```

    flag.StringVar(&metricsCertPath, "metrics-cert-path", "",
        "The directory that contains the metrics server certificate.")
    flag.StringVar(&metricsCertName, "metrics-cert-name", "tls.crt", "The
name of the metrics server certificate file.")
    flag.StringVar(&metricsCertKey, "metrics-cert-key", "tls.key", "The
name of the metrics server key file.")
    flag.BoolVar(&enableHTTP2, "enable-http2", false,
        "If set, HTTP/2 will be enabled for the metrics and webhook
servers")

    opts := zap.Options{Development: true}
    opts.BindFlags(flag.CommandLine)
    flag.Parse()
    ctrl.SetLogger(zap.New(zap.UseFlagOptions(&opts)))

    disableHTTP2 := func(c *tls.Config) {
        setupLog.Info("disabling http/2")
        c.NextProtos = []string{"http/1.1"}
    }
    if !enableHTTP2 {
        tlsOpts = append(tlsOpts, disableHTTP2)
    }

    // Webhook server (keep or remove if you don't need webhooks in the
    UI backend)
    webhookTLSOpts := tlsOpts
    webhookServerOptions := webhook.Options{TLSOpts: webhookTLSOpts}
    if len(webhookCertPath) > 0 {
        setupLog.Info("Initializing webhook certificate watcher using
provided certificates",
            "webhook-cert-path", webhookCertPath,
            "webhook-cert-name", webhookCertName,
            "webhook-cert-key", webhookCertKey)
        webhookServerOptions.CertDir = webhookCertPath
        webhookServerOptions.CertName = webhookCertName
        webhookServerOptions.KeyName = webhookCertKey
    }
    webhookServer := webhook.NewServer(webhookServerOptions)

    metricsServerOptions := metricsserver.Options{
        BindAddress: metricsAddr,
        SecureServing: secureMetrics,

```

```

        TLSOpts:      tlsOpts,
    }

    if secureMetrics {
        metricsServerOptions.FilterProvider =
filters.WithAuthenticationAndAuthorization
    }

    if len(metricsCertPath) > 0 {
        setupLog.Info("Initializing metrics certificate watcher using
provided certificates",
            "metrics-cert-path", metricsCertPath,
            "metrics-cert-name", metricsCertName,
            "metrics-cert-key", metricsCertKey)
        metricsServerOptions.CertDir = metricsCertPath
        metricsServerOptions.CertName = metricsCertName
        metricsServerOptions.KeyName = metricsCertKey
    }

mgr, err := ctrl.NewManager(ctrl.GetConfigOrDie(), ctrl.Options{
    Scheme:      scheme,
    Metrics:      metricsServerOptions,
    WebhookServer: webhookServer,
    HealthProbeBindAddress: probeAddr,
    LeaderElection: enableLeaderElection,
    LeaderElectionID: "04ab6b3a.eirenyx-api",
})

if err != nil {
    setupLog.Error(err, "unable to start manager")
    os.Exit(1)
}

// +kubebuilder:scaffold:builder

// Health / Ready checks for the manager
if err := mgr.AddHealthzCheck("healthz", healthz.Ping); err != nil {
    setupLog.Error(err, "unable to set up manager health check")
    os.Exit(1)
}

if err := mgr.AddReadyzCheck("readyz", healthz.Ping); err != nil {
    setupLog.Error(err, "unable to set up manager ready check")
    os.Exit(1)
}

```

```

apiRouter := mvc.NewRouter(mgr.GetClient())
apiServer := &http.Server{
    Addr:    httpAddr,
    Handler: apiRouter,
}
go func() {
    setupLog.Info("starting HTTP API server", "addr", httpAddr)
    if err := apiServer.ListenAndServe(); err != nil &&
!errors.Is(err, http.ErrServerClosed) {
        setupLog.Error(err, "HTTP server error")
        os.Exit(1)
    }
}()

setupLog.Info("starting manager")
ctx := ctrl.SetupSignalHandler()
go func() {
    <-ctx.Done()
    shutdownCtx, cancel :=
context.WithTimeout(context.Background(), 10*time.Second)
    defer cancel()
    _ = apiServer.Shutdown(shutdownCtx)
}()

if err := mgr.Start(ctx); err != nil {
    setupLog.Error(err, "problem running manager")
    os.Exit(1)
}
}

```